

Can Robustness Against Instruction Format Be Improved Through Adversarial Training In Language Models?

Alfred Muir

BSc Computer Science & Artificial Intelligence
The University of Bath
2024-2025

Can Robustness Against Instruction Format Be Improved Through Adversarial Training In Language Models?

Submitted by: Alfred Muir

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

Instruction fine-tuning is a widely used technique to specialise pre-trained models for specific tasks, and improve zero-shot performance. While a body of work has sought to understand the robustness and generalisation to new tasks, comparatively little has been done to understand how the format and phrasing of inputs used during training affects performance. We use the MAVEN-ERE dataset, and train FLAN-T5-Large on a training set where every example has been paraphrased using twenty different formats. We then evaluate the impact on performance using an evaluation set where the examples are augmented using unseen instruction formats, as well as the consistency of the model answers and representations across the unseen formats. We find that only using one instruction format during training leads to a model which is not robust to other instruction formats. By training on a range of instruction formats, we are able to significantly improve the performance and consistency of the model. However our paraphrase-trained model, while more robust in general, can still experience sudden and sharp changes in performance from minor changes to the format. We also investigate the impact of using different training objectives which encourage consistent representations between paraphrases. However, we find there is little correlation between the consistency of the model’s final vector representation of paraphrased inputs in embedding space, and the answer consistency. We present a hypothesis as to why this might be the case. This work emphasises the importance of training on a wide variety of instruction formats when fine-tuning models.

Contents

1	Introduction	6
1.1	What is NLP?	6
1.2	Instruction Fine Tuning	6
1.3	Adversarial Perturbations	6
1.4	IFT and Paraphrases	7
1.5	Aims and Objectives	8
2	Literature, Technology & Data Survey	9
2.1	Performance on Instruction Tasks	9
2.2	Adversarial Perturbations	10
2.2.1	Adversarial Perturbations in NLP	10
2.2.2	Methods to Generate Natural Language Adversarial Examples	11
2.2.3	Robustness to Variations in Instructions	11
2.3	Transformers & T5 Architecture	13
2.3.1	The Encoder	13
2.3.2	The Decoder	14
3	Methodology	14
3.1	Dataset and Paraphrases	14
3.1.1	Preprocessing	14
3.1.2	Paraphrases	15
3.2	Training Methods	16
3.2.1	Training Data	16
3.2.2	Training Objectives	17
3.3	Evaluation Methods	18
3.3.1	Metrics	18
3.3.2	Visualisations	19
4	Results and Analysis	20
4.1	Evaluation Set Performance	20
4.1.1	Discussion of Performance (Table 3)	20
4.1.2	Discussion of PCA Plots	22
4.2	Model Consistency	25
4.2.1	Discussion of Performance (Table 4)	25
4.2.2	Discussion of Decoder Path Plots	26
4.2.3	Investigation into Evaluation Paraphrase 4	28
4.2.4	Why Does Representational Consistency Not Affect Answer Consistency?	29
5	Future Work and Limitations	30
6	Conclusion	31
A	Paraphrase Formats	38
A.1	Training Paraphrase Formats	38
A.2	Evaluation Paraphrase Formats	38
B	Additional Decoder Trajectory Plots	39
B.1	Additional Examples from Training Set $\mathcal{T}$	39
B.2	Additional Examples from Evaluation Set $\mathcal{V}$	44

Acknowledgements

I would like to thank my supervisor Dr Nadejda Roubtsova for her advice and guidance throughout the project. This was instrumental in developing the project idea and methods, and refining this work. Additionally, I would like to thank my friends for their feedback and support.

1 Introduction

1.1 What is NLP?

Natural Language Processing (NLP) is a field of Artificial Intelligence which seeks to model human language, and enable computers to understand, generate and reason with language. Recent breakthroughs in this field have produced coherent and articulate models, which appear to exhibit impressive knowledge and reasoning abilities. However, models are not always as clever as they seem, having been shown to fabricate information (hallucinate), and exhibit sensitivity to the words used in an input. As language models begin to play a greater role in our collective lives, and especially start playing roles in decision making processes, it is crucial we understand the mechanisms behind their limitations, and how they can be addressed.

1.2 Instruction Fine Tuning

Instruction fine tuning (IFT) is a technique which involves training models to follow instructions, and answer questions. These tasks typically consist of three parts: the instruction, an input, and the desired output. These instructions can cover a broad range of natural language tasks, such as sentiment classification, machine translation, and causal reasoning, examples of which can be found in Figure 1. We focus on the causal reasoning task, which is concerned with establishing relationships between a cause and its effect.

The task of identifying cause and effect can take a number of different forms, each with slightly different framings and objectives. A common form this type of task takes is *event causality identification* (ECI). In this form there is an input (e.g. “The earthquake struck, and the house collapsed”), and an instruction asking whether one event caused another (e.g. “did ‘earthquake’ cause ‘collapsed’?”), with a binary yes/no answer. We focus on this framing of the problem.

Instruction Fine-Tuning Tasks		
Sentiment Classification Instruction: What is the sentiment of the input? Input: “The movie was good” Output: Positive	Translation Instruction: Translate to French. Input: “How are you?” Output: “Comment ça va?”	Causal Reasoning (ECI) Instruction: Did ‘earthquake’ cause ‘collapsed’? Input: “The earthquake struck, and the house collapsed.” Output: Yes

Figure 1: Examples of sentiment classification, translation, and causal reasoning (ECI) IFT tasks.

1.3 Adversarial Perturbations

A desirable property of machine learning models is generalisation: achieving good performance on unseen examples. However, Szegedy et al. (2014) find that even when models *did* generalise to unseen data, they were vulnerable to specific, small changes made to the input. These changes did not alter the semantic meaning of the input, but were sufficient to cause misclassification. Such changes were dubbed ‘adversarial perturbations’ (Figure 2).

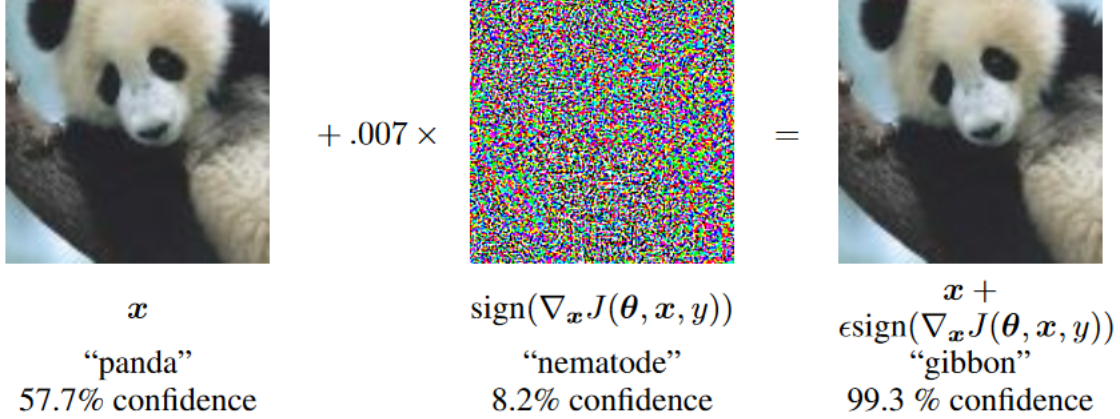


Figure 2: Example of an adversarial perturbation. Applying a perturbation to the input, such that the result appears identical to the original, is sufficient to cause the classifier to misclassify the panda as a gibbon with near certain confidence (Goodfellow, Shlens, and Szegedy, 2015).

While this phenomenon was originally introduced in the visual computing domain, the same idea can be applied to discrete inputs, such as language. Perturbations in language can occur at three different levels, the character-, word-, and sentence-level (Zou, S. Zhang, and Qiu, 2024). Returning to the example input “The earthquake struck, and the house collapsed.”, and example instruction “Did ‘earthquake’ cause ‘collapsed’?”, we can perturb the instruction without altering the semantic meaning at these different levels, as seen in Table 1.

Perturbation Type	Example
Character Level	Did !earthquake¬ cause collapsed; D1d earth quake cause c011apsd? Did Earth quAke cause Collllapsd
Word Level	Did ‘earthquake’ initiate ‘collapsed’? Did ‘earthquake’ result in ‘collapsed’? Did ‘earthquake’ lead to ‘collapsed’?
Sentence Level	Can ‘earthquake’ be said to cause ‘collapsed’? Does the input indicate ‘earthquake’ caused ‘collapsed’? Was ‘collapsed’ a result of ‘earthquake’?

Table 1: Different ways of perturbing the instruction “Did ‘earthquake’ cause ‘collapsed’?”.

1.4 IFT and Paraphrases

We focus on the sentence level perturbation, otherwise referred to as paraphrases. Paraphrases are particularly important in NLP as in downstream tasks, individuals will phrase instructions in different, yet equally valid, ways. Additionally, paraphrases pose a unique challenge due to the diversity of valid perturbed inputs that can be produced: a word’s set of synonyms is only so large, thereby limiting the number of perturbations that can be applied, and character-level perturbed inputs can easily be unperturbed using spell-checkers, or neural networks (Sakaguchi et al., 2017). This limits the variety and effectiveness of examples perturbed by the word- and character-level methods.

Paraphrases of inputs can cause inconsistencies in model predictions. Gao, Ding, et al. (2023) investigate ChatGPT and a number of other language models on the ECI task. Part of their work focussed on evaluating text-davinci-003 (Brown et al., 2020) on the ECI *EventStoryLine* dataset (Caselli and Vossen, 2017) with paraphrases of the instruction. Gao, Ding, et al. (2023) find the model exhibits significant variation in the

outputs, depending on how the instruction is phrased. They use the following phrasings:

- **Standard:** “Given the input: ‘{input}’, is there a causal relationship between ‘{cause}’ and ‘{effect}’?”
- **Counterfactual:** “Given the input: ‘{input}’, if ‘{cause}’ does not happen, will ‘{effect}’ happen?”
- **One Step:** “Given the input: ‘{input}’, is there a one-step causal relationship between ‘{cause}’ and ‘{effect}’?”
- **Trigger:** “Given the input: ‘{input}’, does ‘{cause}’ {TRIGGER} ‘{effect}’?”

Where {TRIGGER} is one of a number of synonyms for ‘cause’. We observe the results of using these phrasing formats in Table 2.

Methods	ESC		
	Pos	Neg	Full
text-davinci-003	74.4	67.7	68.9
- w/ counterfactual	98.5	0.8	16.3
- w/ one-step	44.4	91.7	84.4
- w/ trigger (lead to)	88.3	46.5	53.1
- w/ trigger (give rise to)	90.8	53.7	59.5
- w/ trigger (bring about)	87.1	53.1	58.4
- w/ trigger (result in)	83.3	59.5	63.3
- w/ trigger (in order to)	70.1	60.9	62.4
- w/ trigger (so that)	81.2	59.9	63.3

Table 2: The performance impact of different perturbations of the standard input on text-davinci-003 on the *EventStoryLine* dataset (Gao, Ding, et al., 2023). ‘Pos’ and ‘Neg’ refer to accuracy on the causal and non-causal examples respectively. ‘Full’ refers to the overall accuracy.

This sensitivity is concerning, as it shows that the model’s performance on this IFT task is heavily dependant on the way in which the instruction is phrased. Language models should be invariant under superficial syntactic changes, as paraphrases of an input are semantically identical, and have the same correct answer.

1.5 Aims and Objectives

We seek to answer the following research questions:

- RQ1** To what extent do paraphrased instruction formats impact model performance?
- RQ2** Does training on instructions paraphrased using a diverse range of formats improve model performance on the ECI task, where all of the examples in the evaluation set have been paraphrased using unseen (i.e. not directly used during fine-tuning) instruction formats?
- RQ3** To what extend does training on a variety of formats increase model consistency in both predictions on, and vector representations of, inputs paraphrased using the set of unseen paraphrases?
- RQ4** Does including terms in the training objective to encourage representational consistency improve answer consistency? We use the intuition that more consistent representations lead to more consistent answers.

We choose to train FLAN-T5-Large (Chung et al., 2024), which balances model size (770M parameters) and available compute, and develop a comprehensive training and evaluation framework to understand the impact of paraphrased instruction formats. Our contributions are as follows:

- We introduce a fine-tuning pipeline to improve the consistency and downstream performance of instruction tuned models. We find this pipeline improves the performance of FLAN-T5-Large on an ECI task by up to 8.2% on an evaluation set where each of the examples is paraphrased using one of eight unseen formats, and improves answer consistency on unseen formats by up to 17.4% (relative to the metric scale) compared to fine-tuning on one format.
- We perform a comprehensive analysis on the effect of instruction format on FLAN-T5-Large, evaluating the impact on performance, and consistency of outputs and representations. We adapt metrics and plots for our task to better understand this effect.
- We perform a study to understand the relationship between the final layer representational consistency, and answer consistency, on paraphrases of an input. We do this by analysing the answer consistency achieved by FLAN-T5-Large when trained using loss functions adapted to encourage representational consistency. We find there is little correlation between the consistency of final layer vector representations, and answer consistency, on a *paraphrase set* which uses unseen formats. A paraphrase set is the set of examples formed by taking a unique input, and paraphrasing it using a set of formats.

2 Literature, Technology & Data Survey

In this section, we first discuss the performance of language models on instruction tasks. We then discuss adversarial perturbations, and the impact of paraphrases on performance. Finally, we discuss methods used to improve model performance and robustness.

2.1 Performance on Instruction Tasks

Language models today are articulate, coherent, and appear to show understanding of the world around them. We can evaluate this understanding by querying it using instructions and observing the answer. However, it is unclear whether they truly reason around instructions, or simply memorise patterns and heuristics (Huang and Chang, 2023). Wei, Tay, et al. (2022a) finds some abilities of language models are only present once they reach a certain size. They test a number of models on few-shot prompting tasks, and find accuracy suddenly starts to increase on held-out tasks only once a number of training FLOPs has been reached. This suggests some abilities of language models are only present in larger models, and not present in smaller models. They term these types of abilities as *emergent abilities*, and find they generally start occurring at around 10^{22} FLOPs, or around 10B parameters.

Eliciting reasoning in language models may not just depend on size, but also how a prompt is formed. Conditioning the language model with a few task-specific examples in the prompt improves performance (Brown et al., 2020; Radford et al., 2019). This is known as *few-shot* prompting, and helps the model to understand the desired outcome. Wei, X. Wang, et al. (2022b) find that using a *chain of thought* (CoT) in the prompt significantly improves performance. CoT prompting involves giving the language model a worked example solution to a problem, and asking it to transfer that logic to a new problem. Kojima et al. (2022) go further still, and introduce *Zero-shot-CoT*. This method simply appends “Let’s think step by step” to the end of the prompt, and finds this performs closer to few-shot prompting, and far outperforms the zero-shot equivalent. Further, Lampinen et al. (2022) find that performance can also be improved by providing an explanation of the answer to the model in-context, in the few-shot setting. If these explanations are tuned, performance can be increased even further. That said however, Gao, Ding, et al. (2023) further test text-davinci-003 on the EventStoryLine dataset using few-shot prompting. They find that the model, in all cases they tested (different numbers of positive and negative examples) except one, performed worse than without any in-context learning. The only case where it performed better, was when 4 positive and 48 negative examples were provided.

Rae et al. (2022) find that language models typically perform poorly in logical reasoning tasks. Creswell, Shanahan, and Higgins (2023) seek to improve this by introducing *selection-inference*, a process where language models break down a complex multi-step logical reasoning task into smaller steps, and performing inference on those. The authors find this significantly improves accuracy, as well as providing a reasoning trace, where it can be seen how a model has approached and solved a problem. Madaan et al. (2022) use CODEX, a model trained on code, to perform structured common-sense reasoning. They find that this is an improvement over natural language models. Fu and Khot (2022) have similar findings, and note there is a correlation between models that are trained on code being able to perform CoT reasoning, and those that are not struggling to do so. X. Wang, Wei, et al. (2023) introduce another method to improve CoT reasoning called *self-consistency*. Instead of a model generating a single, ‘best’ reasoning path, it generates multiple diverse chains-of-thought, and selects the majority answer it gets.

Ruis et al. (2023) investigate model understanding of *pragmatic* language, e.g. Person A: “do you want to hang out this weekend?”, Person B: “I have work”. We infer this means ‘no’ as one cannot work and hang out at the same time. This involves *implicature* - implying a meaning beyond what is literally said, as well as common-sense reasoning. They find that base models, even large ones exhibiting emergent behaviours such as GPT-3 175B, perform only just above random in a zero-shot setting. While performance for some models does improve in a k -shot setting, models that have been instruction fine-tuned perform better overall. They find GPT-4 is able to achieve close to human-level performance on this task, even in the zero-shot setting. Bubeck et al. (2023) even go so far as to say that GPT-4 has shown some signs of AGI, due to its human-like abilities in a wide range of tasks.

2.2 Adversarial Perturbations

Szegedy et al. (2014) and Biggio et al. (2013) introduce the idea of adversarial perturbations in the visual computing domain, and discuss how deep neural networks can be highly sensitive to small, imperceptible changes to the input. Szegedy et al. (2014) tested two deep networks trained on the MNIST 2D dataset (LeCun, Cortes, and Burges, 2010), AlexNet (Krizhevsky, Sutskever, and Hinton, 2017) and QuocNet (Le et al., 2012), and observed this phenomenon universally between the models, even though they had been trained on entirely different data. They find training on adversarial examples has a regularising effect. Goodfellow, Shlens, and Szegedy (2015) show that adversarial examples can be explained by linear behaviour in high-dimensional space, and hypothesise neural networks are too linear to resist linear perturbations. They introduce the fast gradient sign method (FGSM), defined as:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)) \quad (1)$$

where x is the input, ϵ is a scaling factor, and $\nabla_x L(\theta, x, y)$ is the gradient of the loss function with respect to the input x . Essentially, this takes the derivative of the loss function with respect to the input, and applies the perturbation which maximises the loss.

Ilyas et al. (2019) explain the pervasiveness of adversarial examples. They show that classifiers are in fact learning highly predictive, yet brittle and non-robust, features of the data. These features are derived from patterns in the data distribution. They also establish the widespread existence of these features in training datasets, and tie them to a misalignment between our (human) notion of robustness, and the geometry of the data.

2.2.1 Adversarial Perturbations in NLP

The finding by Ilyas et al. (2019) is not limited to visual networks in the continuous domain. Indeed, this carries over to language networks in the discrete domain. Because textual data is discrete however, methods used to compromise visual networks cannot directly be applied, though they can be modified such that they can be applied.

Papernot et al. (2016) are first to investigate applying adversarial perturbations to recurrent neural networks (RNNs), by adapting algorithms previously developed to generate adversarial examples for feed-forward and

image networks. They use the FGSM, take derivatives of the loss function with respect to the input embeddings, and iteratively replace tokens in the input with tokens whose embeddings align closest with the perturbed embeddings given by the FGSM. They test an LSTM on a movie review dataset, with the task being to classify the reviews as positive or negative, and achieve 100% error rate on the training set by changing an average of 9.18 words. While the authors don't perform any checks to ensure the resulting sentence's semantic meaning is the same, it can be broadly used to show that much like image networks, language networks are no less vulnerable.

2.2.2 Methods to Generate Natural Language Adversarial Examples

Character- and Word-Level Perturbations

Many present methods which generate adversarial examples at the character and word level. Some present methods which, broadly, use gradients to find the most salient characters in an input, then find the perturbations of these characters which maximises error (Ebrahimi, Lowd, and Dou, 2018; Ebrahimi, Rao, et al., 2018; Heigold et al., 2018). Others apply similar algorithms, but in the 'black box' setting (Gao, Lanchantin, et al., 2018; J. Li et al., 2019). This is one in which the adversary does not have access to the model gradients, so the algorithm instead finds salient inputs indirectly by applying changes to each word, and seeing how much the model output is changed.

Ren et al. (2019) present an algorithm which finds the most salient words in an input, and replaces these with synonyms from WordNet which maximise misclassification. L. Li et al. (2020) attack BERT by ranking words by saliency, then iteratively replacing them with words which are close in embedding space. They also use a sentence similarity checker (Cer et al., 2018) to ensure the perturbed sentence is semantically identical to the original. Likewise, Alzantot et al. (2018) present a genetic algorithm which iteratively substitutes words to maximise misclassification. In both of these papers, the authors further train models using the adversarial perturbations as augmented training data. L. Li et al. (2020) find that while performance on the original dataset remains similar, the model does become more robust to the perturbations. Alzantot et al. (2018) however, find that there is no additional robustness benefit from training on their examples, and don't comment on the resulting accuracy on the original dataset. That said, they only train on 1,000 adversarial examples.

Sentence-Level Perturbations (Paraphrases)

Some present targetted algorithms which generate paraphrases with the intention of deceiving a model. Dong et al. (2017) apply paraphrase rules from the PPDB dataset and WikiAnswers, and back-translation using neural machine translation (this is the process of translating an input to another language, then translating back to English. The aim is that the back-translated response is syntactically different but semantically the same). A question-answer model is then trained using these paraphrases as augmented data, and observe that performance improves across three datasets. Iyyer et al. (2018) propose *Syntactically Controlled Paraphrase Networks* to generate examples. This involves back-translation, then performing a constituency-parse, and training a model to reconstruct the back-translated sentence, given the original sentence and the constituency parse. Training on these examples increases robustness to syntactic variation. In a similar spirit, B. Wang et al. (2020a) uses a tree-based autoencoder. They take a constituency parse of a sentence, which the autoencoder recursively encodes into a latent vector. A decoder is trained to recover the original sentence from the latent representation. Adversarial examples are generated by applying a perturbation to the latent vector which maximises likelihood of fooling the target model, while minimising semantic change.

2.2.3 Robustness to Variations in Instructions

A body of work investigates how language models respond when syntactic changes are made to the instruction. Broadly, language models are shown to be brittle, and have a heavy dependence on instruction presentation.

Some find language models are very sensitive to phrasing, even when not intentionally adversarial. Long, Schuster, and Piché (2024) investigate language models used for building causal graphs in the medical domain. They test GPT-3 in some simple and well know exposure-outcome relationships (alcohol, obesity,

cancer and diabetes), and find that the model was sensitive to the language used to express causality / non-causality. In one case, simply adding “According to medical doctors” to the start of the statements suggesting or not suggesting the existence of a causal edge, improved accuracy by 50% over the baseline. As discussed in Section 1.4, Gao, Ding, et al. (2023) test text-davinci-003 on identifying events as causally related or not, and find that it was sensitive to the exact word used to identify causality (i.e. ‘cause’, ‘lead to’, ‘result in’ etc.), and very sensitive to phrasing. Tu, Ma, and C. Zhang (2023) have similar findings, testing ChatGPT in a causal discovery setting. They find that even asking the exact same prompt on different days lead to different answers and explanations. Hobbhahn, Lieberum, and Seiler (2022) investigate performance of language models in causal settings, for instance: “My car got dirty. I washed the car. Question: Which sentence is the cause of the other?”. They find when the order of the sentences (and even examples provided in-context) are switched, the model tends to perform worse, suggesting it relies on the form and presentation of the question, less so reasoning on it. Z. Li et al. (2024) investigate robustness to *prompt injection* attacks. These are attacks which embed unrelated, potentially harmful, instructions into the prompt. They find that a model’s size and capabilities do not necessarily correlate with robustness to this attack, rather robust models tended to understand the prompt overall, as opposed to focussing on text at the end of the prompt (where the relevant instructions would tend to be).

Others find that large-scale instruction fine tuning improves both robustness and performance. Sanh et al. (2022) train a T5 variant, T5+LM, on a wide range of tasks and datasets, and evaluate on held-out datasets and tasks. They find that the model generalised well and performed much better on the datasets held-out during training, and better than GPT-3 175B (a model 16x its size). They also emphasise the importance of training on a diverse range of prompts, as they found training on more prompts per dataset resulted in better and more robust generalisation to held-out tasks. Similarly, Chung et al. (2024) instruction fine-tune T5 and PaLM variants on the FLAN collection, which consists of a wide variety of different tasks, and includes chain-of-thought data. The resulting models are named FLAN-T5 and FLAN-PaLM. They find doing this significantly increases model performance. They note that fine-tuning on CoT data is critical for reasoning abilities, and that the effectiveness of instruction fine-tuning generalises across models of all sizes, from 80M to 540B parameters.

Despite this however, models can still remain vulnerable to perturbations. Therefore others conduct further studies to find ways of improving robustness. Gu et al. (2023) train and evaluate the 3B and 11B variants of T5 on the Super-NaturalInstru dataset, which contains a total of 154 different tasks. They find the models are vulnerable to natural perturbations of the input, but that robustness improves if demonstrations of how to solve the task are included. Y. Zhang et al. (2024) propose a black-box method to improve robustness. Instead of asking a language model to respond to an instruction in a typical natural language format, they convert this to a code-style format, and ask the model to respond based on that. They find this improves the performance on a perturbed evaluation set. Agrawal et al. (2025) focus on character- and word-level perturbations, and investigate a number of methods to improve robustness to perturbations made by TextFooler (Jin et al., 2020) and DeepWordBug (Gao, Lanchantin, et al., 2018). They find evaluating an ensemble of different instruction formats and selecting the more common answer improves robustness, and that fine tuning the models to minimise the cosine similarity of representations of formats in the middle layer also improves robustness.

Others have approached this problem by introducing terms into the training objective to encourage representational consistency between paraphrased inputs. Gunel et al. (2021) use a joint cross entropy - contrastive loss function to fine-tune RoBERTa-Large. The contrastive element of this loss function aims to bring example representations of the same class closer together, while repelling those belonging to other classes. They find that using this training objective improves both performance and robustness to noisy examples. Yan et al. (2024) introduce ‘COIN’ (**C**ontrastive **I**nstruction tuning), which also uses a joint cross entropy - contrastive loss function. They train Alpaca 7B (Taori et al., 2023) on the FLAN collection of fine-tuning tasks, using 10 manually defined instruction formats, and evaluate using examples from PromptBench which use unseen formats. They find that using this training objective improves accuracy on the evaluation set. Sun, Shaib, and Wallace (2024) evaluate the robustness of a number of models, including FLAN-T5-XL and FLAN-T5-XXL, on a wide range of manually generated unseen instruction formats. They find these signifi-

cantly degrade the performance of the models. They then train the model using three different instruction formats generated using GPT-4, proposing a method to improve the performance on the unseen formats by introducing ‘soft prompt’ parameters (essentially these are ‘novel’ token embeddings appended as a prefix to the input). They then train *only* the soft prompt parameters using a novel cross entropy - KL divergence loss, finding this improves the accuracy of the models on the unseen formats.

Tsipras et al. (2019) investigate the trade off in standard accuracy and robustness in a linear classifier, and find there is a fundamental difference in the feature set of the optimal accurate classifier and the optimal robust classifier, due to the difference in training objectives. They find that it is impossible to have both an accurate and robust classifier in any one task.

2.3 Transformers & T5 Architecture

Raffel et al. (2020) introduce T5, an encoder-decoder transformer trained on a text-to-text training objective. While the training methods used were novel, the architecture used was broadly identical to that introduced by Vaswani et al. (2017) in the original Transformer paper, *Attention is all you Need*. Here, we discuss some parts of the Transformer architecture which are relevant for later sections.

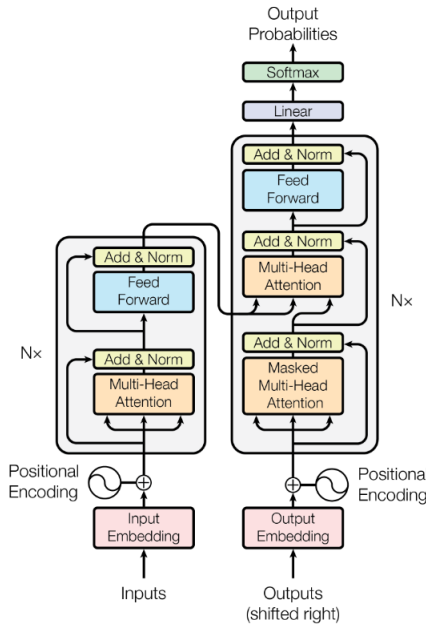


Figure 3: The encoder-decoder Transformer (Vaswani et al., 2017)

2.3.1 The Encoder

Broadly, the encoder takes an input $X \in \mathbb{R}^{L \times d_{model}}$, where L is the sequence length, and d_{model} is the size of the input embeddings, and outputs a matrix of the same dimensions. Essentially, the output of the encoder is a matrix of *contextually rich* embeddings, where each embedding incorporates all the relevant context offered by the tokens around it. It does this using two modules: a multi-head self-attention module, followed by a feed-forward module. In the multi-head self-attention module, the input is projected, for each of keys K , queries Q and values V , h times to a lower dimension d_h . Self-attention, defined for T5 as:

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B\right) \cdot V \quad (2)$$

is performed on each of the lower dimensional projections. These are then concatenated together, and projected back to the d_{model} space. This is done as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \cdot W^O \quad (3)$$

where W^O is a projection matrix $\in \mathbb{R}^{h d_v \times d_{model}}$. The purpose of this module is to incorporate, for each embedding, information from all other embeddings. This is then fed to the feed forward module. This is defined as:

$$\text{FF}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (4)$$

These operations form a single layer. After being applied a number of times, the encoder outputs a matrix, the same dimensions as the input, of contextually rich embeddings.

2.3.2 The Decoder

The decoder uses the encoder output to generate a response. The input to the decoder is the sequence of generated tokens to that point, or a neutral $\langle \text{PAD} \rangle$ token if nothing has yet been generated. The first module is the masked multi-head attention module. The purpose of this is to allow a token to attend to all it's preceding tokens. Since we generate one word responses, this module is not of great interest, as this will only involve the $\langle \text{PAD} \rangle$ token attending to itself.

The next module is a multi-head cross-attention module. This is of far greater interest, as it is where information from the encoder is passed to the decoder. It works in a similar way to the multi-head self-attention layer in the encoder, except the encoder output is projected to become the keys and values, while the decoder output to the point is projected to become the queries. Finally, this output is passed through a feed-forward module, which is again the same as in the encoder.

The output here is a matrix $\in \mathbb{R}^{L_{dec} \times d_{model}}$, where L_{dec} is the length of the decoder output to that point. A new token can be generated by taking the representation of the *most recently* generated token (a vector $\in \mathbb{R}^{1 \times d_{model}}$), and linearly projecting it up to the dimension of the vocabulary using an unembedding matrix $\in \mathbb{R}^{d_{model} \times \|V\|}$, where $\|V\|$ is the length of the vocabulary. This vector is of great interest to us. Because this is mapped directly to the vocabulary vector, this is essentially a vector representation (in $\mathbb{R}^{d_{model}}$ space) of the input. We use this vector to investigate how representationally consistent the model is under various unseen paraphrase formats, and can define loss functions which penalise the model for producing very different representations for paraphrases of an input. This is described in more detail in the next section.

3 Methodology

Here, we establish our training and evaluation framework, which will allow us to answer our research questions, as defined in Section 1.5. We first describe the data we use to train and evaluate on, as well as the paraphrases used, and how these were generated. We then describe how the model was trained, and the various training objectives used to investigate the relationship between representational and answer consistency. Finally, we describe how the model was evaluated, the metrics used, and how we visualised the model outputs to gain deeper insight.

3.1 Dataset and Paraphrases

For this project, we use the MAVEN-ERE dataset (X. Wang, Chen, et al., 2022). This is a large causal reasoning dataset which contains 57,992 causal relations. Since the authors do not publicly release the test set, we instead use the validation set for evaluation. From now, we refer to the MAVEN-ERE validation set as the evaluation set.

3.1.1 Preprocessing

MAVEN-ERE list two types of causal relation, 'CAUSE' and 'PRECONDITION'. We follow the same process as Gao, Ding, et al. (2023), and do not distinguish between the two. This is because these are largely

similar relations, and significantly increases the amount of training data available. We iterate over the causal pairs, and their ‘mentions’ (words used to refer to the event; events may be mentioned more than once). We then find mentions that are within two sentences of each other. We do this, as Gao, Ding, et al. (2023) find that accuracy drops as the lexical distance between events increases. However, we balance this with the need for training data.

We generate one negative example for each positive example, so the datasets are balanced. We generate such examples by randomly sampling two event mentions that are within two sentences of each other, do not refer to the same event, not referred to by the same word (as this is ambiguous), and do not refer to the cause and effect events such that the example would be positive. There were three instances in the training dataset where, because of these conditions, there were no negative examples to select. The end result is a training dataset with 46,305 examples, and an evaluation set with 11,574 examples, following a roughly 80:20 train-evaluation split. A preprocessed example from each of the training and evaluation sets can be found below. We include additional examples, with their decoder trajectory plots (defined shortly), in Appendix B.

Example from Training Set $\mathcal{T}$:

Input: After repelling two attacks, the Texians were unable to fend off a third attack. As Mexican soldiers scaled the walls, most of the Texian fighters withdrew into interior buildings.

Cause: attacks

Effect: withdrew

Label: yes

Example from Evaluation Set $\mathcal{V}$:

Input: This song from 1984 had been the band’s only US Hot 100 number 1, but in his attempt to ‘forget’ the band’s past, Sammy Hagar had refused to sing it at every show when he joined the band in 1985.

Cause: sing

Effect: refused

Label: no

3.1.2 Paraphrases

We define a standard instruction phrasing $\mathcal{P}_{standard}$, which will form a baseline reference. We modify the prompt used by Gao, Ding, et al. (2023) to distinguish the causal direction, such that the overall input is:

“Given the input: ‘{input}’, did ‘{cause}’ cause ‘{effect}’?”

We choose this format as it is a simple, natural way of phrasing the question. This prompt also distinguishes the causal direction, meaning $X \rightarrow Y \neq Y \rightarrow X$. This allows us to generate a broader range of training examples, as for the positive (causal) pairs, we can simply reverse the cause and effect to arrive at a negative (non-causal) example.

While we analysed a number of algorithms for paraphrase generation in Section 2.2.2, we recognise the risks of using such algorithms. Morris et al. (2020) find that some of these algorithms fail to consistently preserve semantics, and commonly introduce grammatical errors. To generate the formats we use ChatGPT 4.5, as we believe this will help eliminate bias and ensure the paraphrase formats are as diverse and varied as possible.

Training Paraphrase Formats

We first define a set of training paraphrase formats $\mathcal{P}_{train}$. These are used to augment examples which the model is trained on. Twenty of these are curated and edited if necessary, some examples are included below. We include all twenty formats in Appendix A.1

1. “Considering the input: ‘{input}’ can we conclude that ‘{cause}’ is responsible for causing ‘{effect}’?”
2. “Given the input: ‘{input}’, can ‘{cause}’ be attributed to causing ‘{effect}’?”
3. “Is ‘{cause}’ a necessary condition for ‘{effect}’ to happen, given the following input: ‘{input}’?”

Evaluation Paraphrase Formats

To evaluate the model, we use a separate set of evaluation paraphrases $\mathcal{P}_{eval}$. We use these to evaluate how well the model generalises to unseen input formats. We again use ChatGPT 4.5 to generate further paraphrases, eight of which are curated and edited if necessary. We include three examples, the full list of eight formats can be found in Appendix A.2

1. “Taking the statement ‘{input}’ into account, is ‘{cause}’ the event that made ‘{effect}’ occur?”
2. “If ‘{input}’ holds true, can we definitively state that ‘{effect}’ was brought about by ‘{cause}’?”
3. “Does ‘{cause}’ directly explain why ‘{effect}’ took place, based on the information: ‘{input}’?”

3.2 Training Methods

We define a comprehensive training program to answer the research questions. Broadly, we can break the program down into two distinct parts.

First, to understand the impact of using alternative paraphrase formats, we train the model on the training set where each example uses the standard phrasing format $\mathcal{P}_{standard}$. We then evaluate the model on the evaluation set, where each example uses $\mathcal{P}_{standard}$, and compare that to the evaluation set where each example is randomly augmented with a paraphrase format from $\mathcal{P}_{eval}$. From now, we refer to the training set augmented with $\mathcal{P}_{standard}$ and $\mathcal{P}_{train}$ as $\mathcal{T}_{standard}$ and $\mathcal{T}_{paraphrase}$ respectively, and the evaluation set augmented with $\mathcal{P}_{standard}$ and $\mathcal{P}_{eval}$ as $\mathcal{V}_{standard}$ and $\mathcal{V}_{paraphrase}$ respectively. We use these results to answer RQ1.

Next, we train the model using $\mathcal{T}_{paraphrase}$, to understand the performance impact on $\mathcal{V}_{paraphrase}$. Parallel to this, we also investigate whether using training objectives which encourage representational consistency between paraphrase formats can improve answer consistency and performance (Gunel et al. (2021); Yan et al. (2024); Sun, Shaib, and Wallace (2024)). We evaluate again on $\mathcal{V}_{standard}$ and $\mathcal{V}_{paraphrase}$, using these results to answer RQ2 and RQ4. We also evaluate the consistency of these models using the metrics and visualisations defined in Section 3.3. We use these results to answer RQ3.

We term each stage of our training process as a training stage (TS). At a high level, our training process is as follows:

TS0 Begin with the ‘untrained’ FLAN-T5-Large

TS1 Train on $\mathcal{T}_{standard}$ using CE loss

TS2 Take the trained model from TS1, and further separately train the model using $\mathcal{T}_{paraphrase}$ with the following loss functions:

TS2.1 Cross Entropy (CE)

TS2.2 Cross Entropy - Mean Squared Error (CE-MSE)

TS2.3 Cross Entropy - Cosine Similarity (CE-CS)

TS2.4 Cross Entropy - Contrastive (CE-CT)

We define the additional loss functions in TS2 in detail in Section 3.2.2.

3.2.1 Training Data

We train FLAN-T5-Large using the MAVEN-ERE training dataset. $\mathcal{T}_{standard}$ is the full training dataset, where each example uses $\mathcal{P}_{standard}$. $\mathcal{T}_{paraphrase}$ is created by sampling a random subset of examples from the MAVEN-ERE training dataset, such that this subset is $1/20^{\text{th}}$ the size of the full training dataset, and augmenting each of these examples with *all* of the twenty paraphrases in $\mathcal{P}_{train}$. This is so $\mathcal{T}_{standard}$ and $\mathcal{T}_{paraphrase}$ are the same size.

3.2.2 Training Objectives

Categorical Cross Entropy (CE)

Cross entropy is a commonly used training objective for classification tasks. Categorical cross entropy is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log(p_{i,j}) \quad (5)$$

where N is the number of examples, C is the number of classes, $y_{i,j}$ is the one-hot encoded true label, and $p_{i,j}$ is the output probability for a class j for an example i (Terven et al., 2024).

While generally effective, this loss function only incentivises the model to answer correctly, with no explicit incentive to ensure the vector representations of paraphrased inputs are in any way similar.

Cross Entropy - Mean Squared Error Loss (CE-MSE)

We use a joint cross entropy - mean squared error (CE-MSE) loss, adapted for our task, to investigate the impact of introducing an incentive for the model to map paraphrased inputs to similar locations in $\mathbb{R}^{d_{model}}$ space. This training objective is inspired by a finding by Sun, Shaib, and Wallace (2024). They find that after soft-prompt alignment, the average distance between the vector representations of seen and unseen paraphrase formats reduces, and suggest this may be a mechanism of improvement from such alignment. We investigate whether incentivising the model to bring representations closer, results in a higher performing and more consistent model.

To implement the MSE part of the loss, we first find all of the unique inputs in the batch (as each unique input is paraphrased twenty times). We then group the representations of the paraphrased inputs, and iterate over the groups, discarding groups containing only one representation. At each iteration, we find the mean vector of the group, and sum the average mean squared error between each of the representations in the group, and the mean vector. Lastly, we average over the number of groups (which contained more than one representation). This is shown in Algorithm 1. We define the loss as the weighted sum of the two parts:

$$\mathcal{L}_{CE-MSE} = \lambda \mathcal{L}_{CE} + (1 - \lambda) \mathcal{L}_{MSE} \quad (6)$$

Where λ is a weighting hyperparameter.

Algorithm 1 Implementation of MSE part of CE-MSE Training Objective

Require: representations of examples in random batch B

```

1:  $L \leftarrow 0$  ▷ Initialise total loss
2:  $g \leftarrow 0$  ▷ Initialise count of non-single paraphrase groups
3:  $U \leftarrow$  unique input IDs
4: for each  $id$  in  $U$  do
5:    $P \leftarrow$  examples in  $B$  which have unique ID  $id$ 
6:   if  $|P|$  greater than 1 then
7:      $v \leftarrow \text{mean}(P)$ 
8:      $L += \text{MSE}(P, v)$  ▷ Find MSE of all representations in  $P$  with mean vector  $v$ 
9:      $g += 1$  ▷ Increment  $g$ , since a non-single group has been processed
10:  end if
11: end for
12:  $L \leftarrow L/g$  ▷ Average  $L$  over non-single groups which contributed to the loss
13: return  $L$ 
```

Cross Entropy - Cosine Similarity Loss (CE-CS)

Following the same process as the CE-MSE loss, we swap the MSE component of the loss to a cosine similarity (CS) component, by measuring $(1 - \text{CS})$ of each representation to it's groups mean vector as opposed to MSE. We do this to investigate whether making the paraphrased inputs more directionally aligned is more

effective than bringing them closer together. We also define this loss as the weighted sum of the parts:

$$\mathcal{L}_{CE-CS} = \lambda \mathcal{L}_{CE} + (1 - \lambda) \mathcal{L}_{CS} \quad (7)$$

Cross Entropy - Contrastive Loss (CE-CT)

Finally, we investigate using a joint cross entropy - contrastive (CE-CT) loss, following from the success of others in using this (Gunel et al. (2021); Yan et al. (2024)). The contrastive part of this loss differs from the MSE and CS parts of the previous losses, in that it does not specifically look at the paraphrase groups. Rather, it encourages the model to bring representations of examples belonging to one class closer together, while simultaneously repelling those belonging to the other class. The aim is to indirectly bring paraphrase representations closer together, and bring the positive and negative classes respectively closer together, while repelling both classes away from each other in a way which maintains and maximises classification accuracy (from the CE part of the loss). The supervised contrastive loss is defined by Khosla et al. (2020) in (8). We use the PytorchMetricLearning (Musgrave, Belongie, and Lim, 2020) implementation of this loss function.

$$\mathcal{L}_{out}^{sup} = \sum_{i \in I} \mathcal{L}_{out,i}^{sup} = \sum_{i \in I} -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{a \in A(i)} \exp(z_i \cdot z_a / \tau)} \quad (8)$$

Where $\tau > 0$ is an adjustable temperature parameter, which controls how strongly classes are clustered. Lower values of τ encourage stronger clustering. The overall loss function is:

$$\mathcal{L} = \lambda \mathcal{L}_{CE} + (1 - \lambda) \mathcal{L}_{SupCon} \quad (9)$$

3.3 Evaluation Methods

We now explain the methods we use to evaluate the model. Broadly, we use a mixture of metrics and visualisations to explore the model after training.

3.3.1 Metrics

To answer RQ1 and RQ2, we use accuracies on the evaluation sets to understand the model performance. To answer RQ3 and RQ4, we use consistencies to understand how similarly the model is treating an input's *paraphrase set* - the set formed by paraphrasing a unique input using a number of formats.

Overall Accuracy

We use overall accuracy on the evaluation sets as one of the measures used to evaluate model performance. This is simply the proportion of answers it predicted correctly.

Accuracy on Positive and Negative Examples

As explored in Section 1.4, Gao, Ding, et al. (2023) found text-davinci-003 experienced a significant discrepancy in the accuracy achieved on the positive (causal) and negative (non-causal) examples. We use this accuracy measure to help us gain further insight into how the model performs on examples from the two classes.

Answer Consistency

Taking inspiration from L. Wang et al. (2020b), we define the *answer consistency* of a paraphrase set as the count of the mode prediction, divided by the length of the set. This metric is not impacted by the correctness of predictions, instead measuring the extent to which model outputs are changed by applying different instruction formats:

$$\text{answer_consistency}(P) = \frac{\text{count}(\text{mode}(P))}{|P|} \quad (10)$$

where P is the set of predictions on a paraphrase set.

Representational Consistency

We also look at the *representational consistency* of a paraphrase set, to investigate the relationship between the consistency of the vector representations and the consistency of the predictions. To evaluate this, we first obtain the vector representation of each of the inputs in the paraphrase set, and find the mean vector of these representations. We then find the average of the Euclidean distance, and separately the cosine similarity, of each of the representations with the mean vector (similar to the CE-MSE and CE-CS loss functions). This indicates how directionally similar, and how close, the representations are in $\mathbb{R}^{d_{model}}$ space.

3.3.2 Visualisations

We use two types of visualisation to aid us in the evaluation of the model. These are *PCA plots*, and *decoder trajectory plots*. We use these to gain a qualitative insight into the model before and after training.

PCA Plots

These plots are used to visualise the model outputs after each training stage. To create these plots, we take a random subset of 2048 examples from each of $\mathcal{V}_{standard}$, $\mathcal{V}_{paraphrase}$, and $\mathcal{T}_{standard}$ or $\mathcal{T}_{paraphrase}$ (whichever is applicable), and obtain their representations. We then perform PCA with two components on these vectors, to project them from 1024 (d_{model}) dimensions to 2 dimensions. This allows us to visualise the clustering patterns of the data points on a 2D plot. We choose not to normalise the representations, as it allows us to better understand what is being represented to the unembedding matrix. Testing with normalised representations yielded plots which were difficult to read, with dense, overlapping clusters. We then plot the 2D projection of the decision boundary, to visualise how the model is classifying the data. This is done by finding the difference between the weight rows in the unembedding matrix corresponding to the ‘yes’ and ‘no’ logits, and projecting this onto the 2D plane formed by the PCA projection of the data.

Decoder Trajectory Plots

To visualise the trajectory of an input through the decoder, we introduce the *decoder trajectory plot*. In our case, we focus on the values of the ‘yes’ and ‘no’ logits in the vocabulary vector. Essentially, this plot is used to visualise the level of confidence the model has in each of these two logits as the input is processed by the decoder.

The process for generating this plot begins by obtaining the representation of the input at each of the decoder layers. For each of the representations, we take the dot product with the weights in the unembedding matrix corresponding to the ‘yes’ and ‘no’ logits; this tells us the value the model would place on those logits at a given layer. The difference in these values then indicates the model confidence in either of the logits. However, due to the way in which FLAN-T5 normalises activations, these values can get extremely large. Therefore we find the ℓ_2 normalisation of the two values first, before taking the difference. We then do this for each format in a paraphrase set of an input, and plot each of the trajectories. Intuitively, a difference of 0 indicates it believes ‘yes’ and ‘no’ to be equally likely, with a greater positive difference showing it believes the answer is more likely to be ‘yes’, and a greater negative difference showing it believes the answer is more likely to be ‘no’. The process for generating the plot is defined in Algorithm 2.

Algorithm 2 Decoder Trajectory Plot Generation

Require: output vectors V of input at each decoder layer

```
1:  $G \leftarrow []$ 
2: for each vector  $v$  in  $V$  do
3:    $vy \leftarrow$  dot product of  $v$  with unembedding weights  $y$  corresponding to ‘yes’
4:    $vn \leftarrow$  dot product of  $v$  with unembedding weights  $n$  corresponding to ‘no’
5:    $vy_2, vn_2 \leftarrow (vy, vn)_2$ 
6:    $g \leftarrow vy_2 - vn_2$ 
7:   APPEND( $G, g$ )
8: end for
9: return  $G$ 
```

4 Results and Analysis

We use a batch size of 64, a learning rate of 5×10^{-4} , and the AdamW optimiser. The embedding of each token is a vector $\in \mathbb{R}^{1024}$. In TS1, we train FLAN-T5-Large using these hyper-parameters for 20 epochs. Then for TS2, we train for a further 10 epochs, again using the same hyper-parameters. We train these models using a single Nvidia A100 GPU. For the CE-MSE (TS2.2) and the CE-CS (TS2.3) loss functions, we set $\lambda = 0.9$. For the CE-CT (TS2.4) loss function, we set $\lambda = 0.95$. We select the values of λ such that the non-CE parts of the loss functions do not dominate, and cause the model to over-optimize for representational consistency, since accuracy is our measure of performance. Especially for the CE-CT loss, we found empirically that smaller values for λ caused the contrastive element of the loss, which is numerically much larger than the CE part, to dominate. This resulted in poor predictive performance.

For τ , we tested values of [0.3, 0.05, 0.01], finding $\tau = 0.3$ generally performed the best. However, we include the findings for $\tau = 0.05$ and $\tau = 0.01$ as TS2.4.1 and TS2.4.2 respectively, as we believe these yielded interesting results.

4.1 Evaluation Set Performance

Training Regime	V_{standard}			$V_{\text{paraphrase}}$		
	Pos	Neg	Overall	Pos	Neg	Overall
TS0	85.1	40.6	62.9	85.7	35.9	60.8
TS1	84.0	78.6	81.3	45.2	85.3	65.2
TS2.1 (CE)	82.3	76.4	79.4	71.7	74.8	73.3
TS2.2 (CE-MSE)	77.5	78.6	78.0	68.8	76.2	72.5
TS2.3 (CE-CS)	82.0	75.3	78.7	71.9	73.7	72.8
TS2.4 (CE-CT)	81.5	77.3	79.4	72.7	74.0	73.4
TS2.4.1 (CE-CT)	81.1	77.2	79.2	71.8	74.8	73.3
TS2.4.2 (CE-CT)	84.1	73.5	78.8	73.3	71.9	72.6

Table 3: Positive, Negative, and Overall Accuracy (%) on the evaluation set augmented with the standard phrasing (V_{standard}) and the evaluation paraphrases ($V_{\text{paraphrase}}$), achieved by using each of the different training stages.

4.1.1 Discussion of Performance (Table 3)

TS0 We first analyse the results achieved by the ‘untrained’ FLAN-T5-Large. We observe similar results on both of the evaluation sets, with an overall accuracy of 62.9% and 60.8% achieved on V_{standard} and $V_{\text{paraphrase}}$ respectively. We see that in both cases, the accuracy on positive examples is significantly higher than that on the negative examples. We believe this is because in natural language, non-causality between events is generally implied by omission, resulting in a significant class imbalance between causal and non-causal examples observed during pre-training. Therefore, the model will have observed many more causal examples than non-causal examples, and be more likely to answer ‘yes’. As for the similar performance achieved on both datasets, we believe this is due to the wide range of tasks it has previously been trained on, resulting in a robust model despite the poor predictive performance.

TS1 In this training stage, we observe a significant performance increase on V_{standard} , with the overall accuracy increasing from 62.9% to 81.3%, though that observed on $V_{\text{paraphrase}}$ is modest in contrast, increasing only from 60.8% to 65.2%. Essentially, by using unseen paraphrase formats instead of the standard phrasing, the model performance is substantially reduced, from 81.3% to 65.2%. We believe this shows the model has overfit to the standard phrasing, and does not generalise when the instruction is phrased differently to that it was trained on.

We also observe a significant change in positive and negative accuracy compared to TS0. By training on $\mathcal{T}_{standard}$, we are able to significantly increase the accuracy on the $\mathcal{V}_{standard}$ negative examples, from 40.6% to 78.6%. We attribute this to the increase in exposure to negative examples, where these have likely been under-represented in the pre-training data. As for the positive and negative accuracy on $\mathcal{V}_{paraphrase}$, we observe a slight increase in performance of the negative examples, however there is a significant decrease in performance on the positive examples. It is unclear why the positive example accuracy is so heavily affected, while the negative example accuracy increases. We leave establishing the mechanism of effect of unseen paraphrase formats on a model trained using only one paraphrase format as future work.

While the model performance on $\mathcal{V}_{standard}$ increases in TS1, the robustness decreases. We observe the decrease in robustness by comparing the difference in overall accuracy achieved on the two evaluation sets by TS0 and TS1. While TS0 performs poorly overall, it is robust, with the unseen paraphrase formats $\mathcal{P}_{eval}$ decreasing overall performance by only 2.1%. In contrast to TS1, the unseen paraphrase formats decrease overall performance by 16.1%. This shows that while model performance increased on $\mathcal{V}_{standard}$, this increase did not generalise to the formats used in $\mathcal{V}_{paraphrase}$, with the formats used significantly decreasing performance.

TS2 When trained on paraphrases of the instruction format, regardless of training objective, we observe a significant increase in model performance on $\mathcal{V}_{paraphrase}$ compared to TS1, by as much as 8.2%. We believe this is because the model has learned more generalised features, that aren't as sensitive to the format used, and is therefore more robust. This highlights the importance of training language models on diverse instruction formats. For all models in TS2 however, we observe a slight decrease in performance on $\mathcal{V}_{standard}$ compared to TS1. This is expected based on the finding by Tsipras et al. (2019), where the models have traded accuracy for robustness. The addition of mean squared error and cosine similarity training objectives (TS2.2, TS2.3) have a negative impact, with the overall accuracies of both on $\mathcal{V}_{paraphrase}$ being slightly lower than that of TS2.1. While adding a contrastive element to the loss has a slightly positive impact for $\tau = 0.3$, reducing τ to 0.01, which more strongly encourages representational consistency, also has a slightly negative impact. These findings could also be attributed to the finding by Tsipras et al. (2019), but only if representational consistency is not contributing to an increase in accuracy.

Intuitively, we might expect that training on a variety of paraphrase formats *and* training to make the paraphrase representations more consistent would be a net benefit for performance on $\mathcal{V}_{paraphrase}$, or at least not a net loss. However, we find this is not the case, as is evidenced by the performance of TS2.1 and TS2.4 being consistently slightly better than the models which encourage more similar representations (TS2.2, TS2.3, TS2.4.2), which we further discuss in Section 4.2.1. We believe accuracy has been sacrificed in favour of making the representations more consistent. Because accuracy must be sacrificed to achieve this however, this would indicate that representational consistency is not benefitting accuracy.

4.1.2 Discussion of PCA Plots

TS0 (Figure 4) We immediately see that across all the datasets, the examples are distributed almost randomly, with no clear structure. We see significant overlap of the two classes, especially in (a) and (b). Though FLAN-T5-Large has been fine-tuned on causal reasoning examples in the FLAN collection, it is clear this does not generalise to the MAVEN-ERE examples. We also see that for (a) and (b), two dimensions only explains 39.42% and 40.04% of the total variance, whereas this is significantly higher for (c), with 72.71% of the data variance explained by two dimensions. As well in (c), we also observe a different cluster shape to (a) and (b). We attribute both of these observations to the paraphrased examples in (c) varying in different ways to the standard phrased examples in (a) and (b); of course in (a) and (b), the format remains constant (with no variance), while in (c) both the format and content vary.

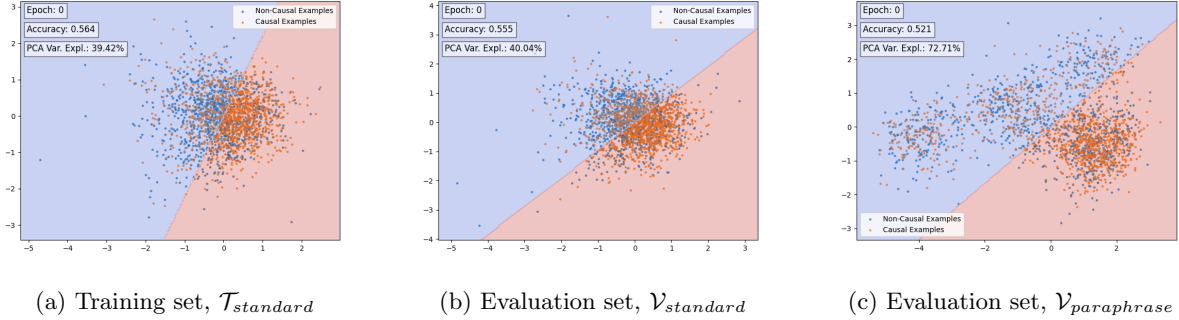


Figure 4: Un-normalised 2D PCA projection of a subset of vector representations from **TS0** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights. We include $\mathcal{T}_{standard}$, even though the model is of course not yet trained, for comparison to TS1.

TS1 (Figure 5) In contrast to the visualisations in Figure 4, training on $\mathcal{T}_{standard}$ produces representations with far more structure. While we observe almost perfect separation of the training data in (a), this is not quite the case on the evaluation data in (b). Despite these examples using the same instruction format as was trained on, there is still a fairly significant amount of overlap between the classes nearer to the decision boundary, though these examples do maintain a similar structure to (a). This indicates some overfitting to the training data content, where the model hasn’t been able to fully generalise to unseen examples. As for (c), we observe a deterioration of this structure. We also see that the PCA variance explained by two dimensions has increased substantially in (a) and (b) compared to TS0 (increasing from 39.42% and 40.04%, to 79.21% and 76.14% respectively), but decreases slightly in (c), to 64.45%. We attribute the deterioration of the structure, and the different PCA variance capture in (c), again to the paraphrased examples varying in ways which which the model isn’t capturing having not been trained on a variety of formats.

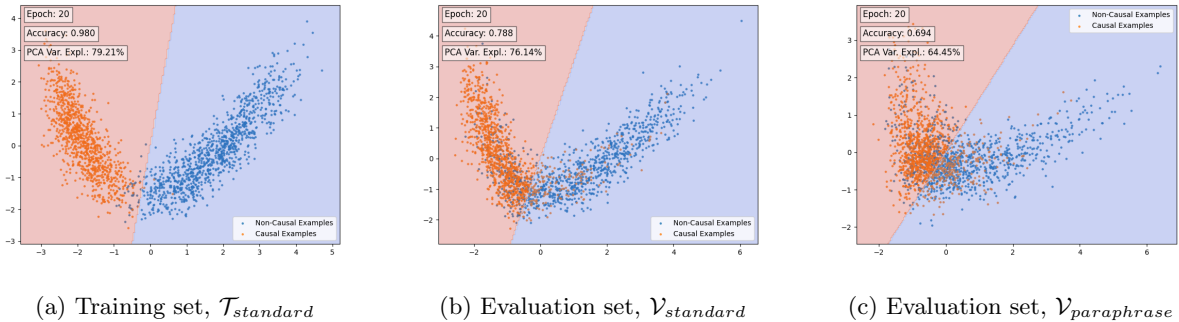


Figure 5: Un-normalised 2D PCA projection of a subset of vector representations from **TS1** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights.

TS2.1 (Figure 6) Training using paraphrases produces a different structure to that of TS0 and TS1. Two dimensions now describes almost all of the data variance across all the datasets (greater than 99%), with the variance explained by (c) this time increasing in line with (a) and (b). We attribute this to the representations of the inputs becoming lower rank from training on $\mathcal{V}_{paraphrase}$. We believe this is due to the increase in variance of the training data, which allows the model to better capture the ways in which the data varies. Most importantly however, is that the clustering of (b) and (c) are much more similar than (b) and (c) of Figure 5 (TS1), showing that the model is less affected by the different instruction formats used in $\mathcal{V}_{paraphrase}$. We make this observation for all subsequent training stages.

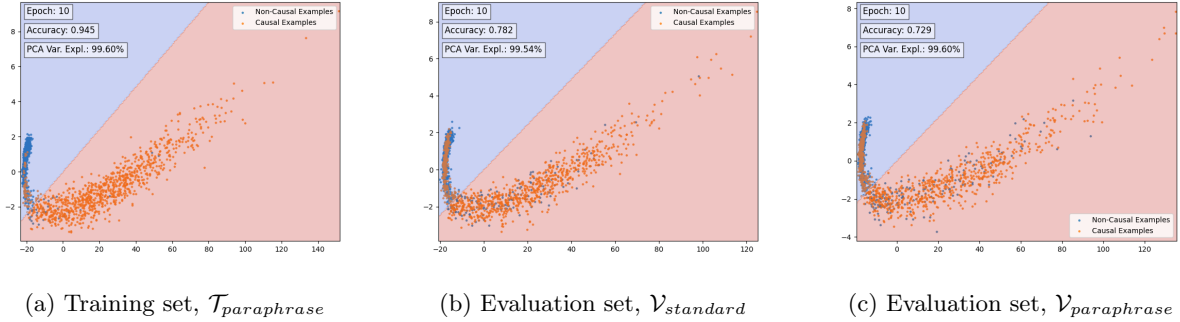


Figure 6: Un-normalised 2D PCA projection of a subset of vector representations from **TS2.1** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights.

TS2.2 (Figure 7) Using the CE-MSE loss function produces a different structure to that of the CE loss in Figure 6 (TS2.1), more similar to that in Figure 5 (TS1). We observe too that while less variance is captured across all datasets compared to TS2.1, the indicated accuracy achieved remains very similar. We also note that the clustering of the two classes is far more even in all datasets, in contrast to the very clustered negative examples, and spread out positive examples, in TS2.1. By consulting the PCA variance scale, especially on the x-axis, we see that the overall data clustering is tighter than that in TS2.1 across all datasets, with the scale broadly being between -2 and 2, as opposed to -20 and 120. This shows that the addition of the MSE part of the loss function results in far more tightly clustered representations.

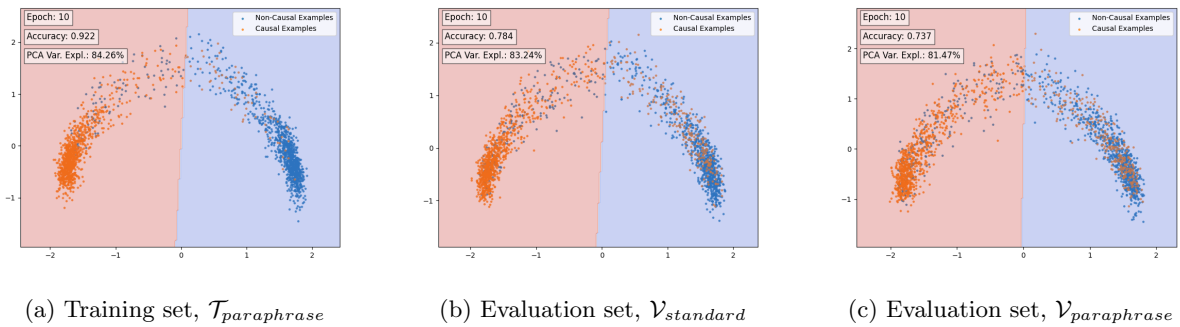


Figure 7: Un-normalised 2D PCA projection of a subset of vector representations from **TS2.2** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights.

TS2.3 (Figure 8) We again see a different structure produced using this loss function. We observe that the amount of variance explained has increased from TS2.2, especially on the evaluation sets, from 84.26%, 83.24%, and 81.47% respectively, to $\sim 93.5\%$ across all datasets. This suggests optimising for cosine similarity produces representations which map more closely to a 2D subspace than those produced by mean squared error. We also observe classes which appear to cluster directionally, forming lines as opposed to clouds. We attribute this to the cosine similarity objective, which encourages directional alignment.

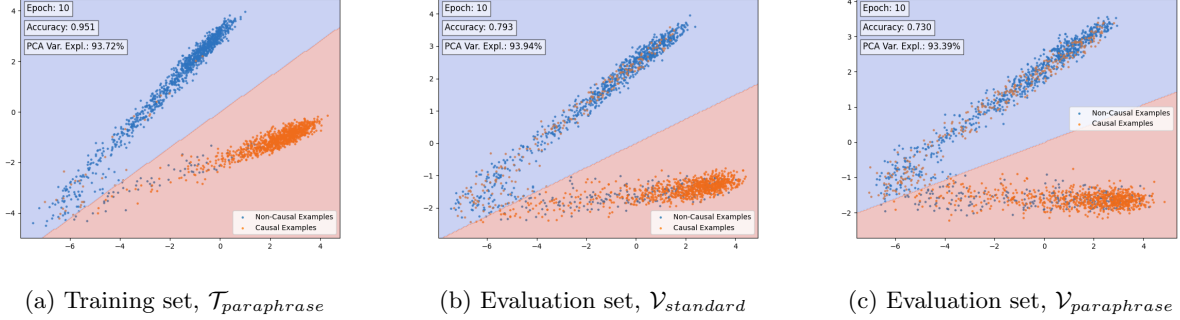


Figure 8: Un-normalised 2D PCA projection of a subset of vector representations from **TS2.3** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights.

TS2.4 (Figure 9) We see the classes form a unique, non-convex structure, not seen produced by the other models. The manifold formed is slimmer, showing the contrastive objective to cluster the classes, as opposed to the paraphrase sets. However, the data varies far more along the x-axis than seen in the other clustering objectives (TS2.2 and TS2.3). This is likely product of the other contrastive objective, which pushes the classes away from each other. Additionally, we seen the PCA variance explained by two dimensions has increased compared to TS2.2 and TS2.3, on par with that seen in TS2.1. This could be due to the broader application of the contrastive element, in that it applies to the positive and negative classes as a whole. This, in contrast to the more local application of the CE-MSE and CE-CS training objectives, which apply only to the paraphrase sets.

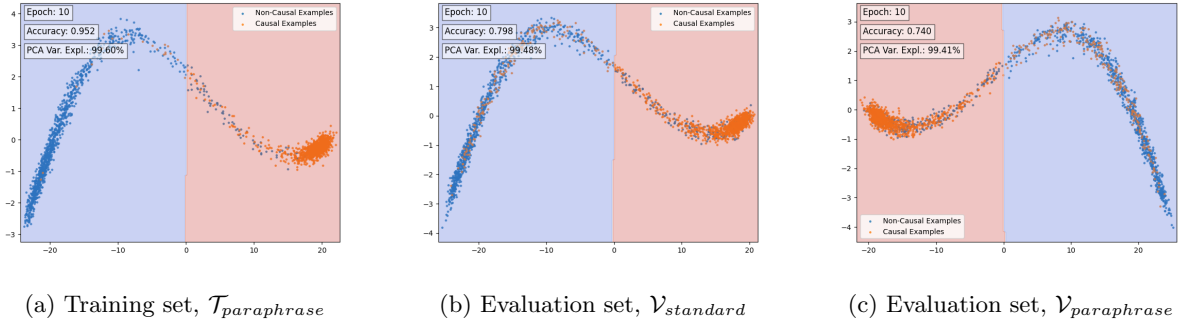


Figure 9: Un-normalised 2D PCA projection of a subset of vector representations from **TS2.4** for each of the relevant datasets, with a projection of the decision boundary formed by the ‘yes’ and ‘no’ logit weights.

4.2 Model Consistency

To measure the model consistency, we take the evaluation set and augment each example with *every* evaluation paraphrase $\mathcal{P}_{eval}$. We then iterate over the unique examples, and apply the evaluation metrics to each paraphrase set, averaging across the sets.

Training Regime	Avg. Dist. ($\downarrow$)	Avg. Cos. Sim. ($\uparrow$)	Ans. Consistency ($\uparrow$)
TS0	2.25	0.9959	86.0%
TS1	1.10	0.9971	81.0%
TS2.1 (CE)	7.51	0.9918	88.9%
TS2.2 (CE-MSE)	0.80	0.9747	88.9%
TS2.3 (CE-CS)	1.34	0.9984	88.7%
TS2.4 (CE-CT)	6.01	0.8296	89.1%
TS2.4.1 (CE-CT)	2.33	0.9688	89.7%
TS2.4.2 (CE-CT)	2.03	0.9933	88.1%

Table 4: Representational and Answer consistency scores achieved using each training method. Scores are averaged across all the paraphrase sets. $\uparrow$ indicates higher is better, $\downarrow$ indicates lower is better.

4.2.1 Discussion of Performance (Table 4)

TS0 We first observe that prior to fine-tuning (TS0), the model is moderately consistent, achieving 86.0% answer consistency. We believe this is due to the diversity of IFT tasks it had previously been trained on, and the natural diversity of presentations and instruction formats it would have encountered as a result.

TS1 Here, we observe a decrease in answer consistency, from 86.0% to 81.0%, but an *increase* in average cosine similarity, and a *decrease* in average distance, between paraphrase representations. Peculiarly, despite example representations becoming more invariant (on average) under paraphrasing, this is not translating to greater consistency in the model answers. This would suggest that vulnerability to unseen paraphrase formats is not dependent on the consistency of final representations. Interestingly too, is that the representations can be almost directionally identical (due to the high average cosine similarity of 0.9971), and yet only lead to the same answer being given 81.0% of the time.

As discussed in Section 4.1, intuition might suggest that the more similar representations of an input are, the less likely it is that these would get mapped to different answers. However, the results show that greater representational consistency does not lead to greater answer consistency. We suggest a hypothesis for this in Section 4.2.4.

TS2 We observe that training on paraphrases of an input leads to higher answer consistency, with all of the models from TS2 experiencing a increase in answer consistency from TS1 and TS0. TS2.4.1 produced the most answer consistent model, achieving a score of 89.7%, an increase of 17.4% from TS1 (relative to the metric scale). The least answer consistent model is TS2.4.2, despite the only difference between TS2.4.2 and TS2.4.1 being that TS2.4.2 encourages stronger clustering. Of note, we observe that TS2.1 produces paraphrase representations which are less similar, and much further apart, than those in TS1, achieving much higher answer consistency despite this. TS2.2 produces representations which are much closer together than TS2.1, and yet achieves the same answer consistency. TS2.3 produces representations with similar representational consistency scores to TS1, achieving 1.34 and 1.10 average distance, and 0.9971 and 0.9984 average cosine similarity respectively, but achieving a much higher answer consistency (88.7%, up from 81.0%). Most prominently, TS2.4 produces representations which are not only far apart, but are significantly less directionally similar than all the other models, and yet achieves the second highest answer consistency. While TS2.4.1 produces representations which are more consistent than TS2.4 in both distance and cosine similarity measures, they are not as consistent as produced by TS2.4.2, and yet achieves the highest answer consistency by a comfortable margin. These observations provide further evidence that answer consistency

does not depend on representational consistency.

4.2.2 Discussion of Decoder Path Plots

Example 1

Input: “The cyclone initially weakened quickly after moving inland, falling to Category 1 status early the next day.”

Cause: “moving”

Effect: “falling”

Label: yes

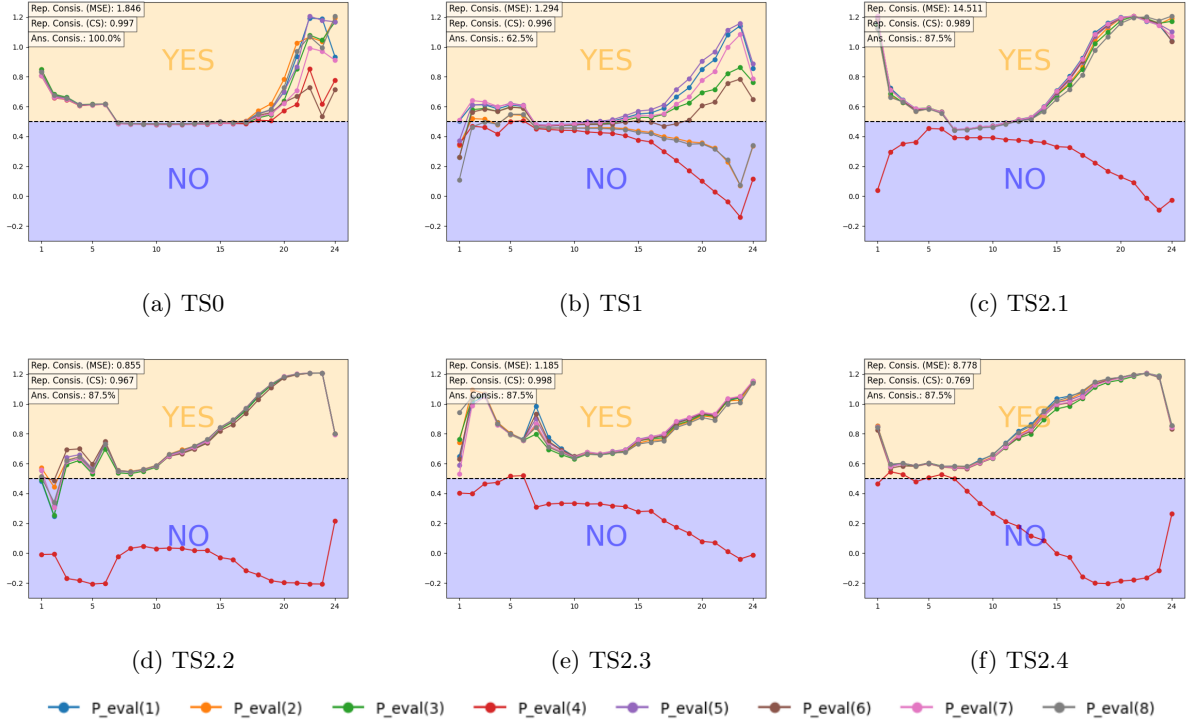


Figure 10: Decoder trajectory plots of the different training stages for Example 1. Each trajectory corresponds to a paraphrase format in $\mathcal{P}_{eval}$. Decoder layer number is represented on the x-axis. Normalised difference is represented on the y-axis.

We observe in (a) that TS0, while achieving 100% answer consistency, appears to be the most ‘noisy’, in that there is lots of volatility in the magnitude of the confidence in the ‘yes’ and ‘no’ logits, especially in the later layers. In the earlier layers however, the paraphrases are treated almost identically. This is likely because the relevant features needed to respond to the instruction are not being activated until layer 17. When these features *are* activated, the difference in format is sufficient to cause each of the examples to follow rather different trajectories, even though they all end up being correctly classified.

However, we observe TS1 (b) to be the most vulnerable to format differences. Firstly, we see that the layer at which the examples begin to diverge from each other is much earlier than in TS0 (a), at layer 7. We also see a lot more variation in the trajectories, even after the first layer. This shows the model has chosen to represent very different features having been further fine-tuned on the MAVEN-ERE dataset. However, the feature selection causes it to be vulnerable to differences in instruction format.

We observe some clear trends in the TS2-trained models. Firstly, we see that across the models, seven

of the eight paraphrases are being treated almost identically, most prominently in TS2.2, TS2.3, and TS2.4. This shows that the joint training objectives do help the model become more representationally invariant to paraphrases, as these seven formats follow a much more similar trajectory than in TS2.1. However, we observe that the confidence in either of the two logits does not seem to correlate with the representational consistency metrics. Taking TS2.3 and TS2.4 (e, f) for instance, we see there is one format in both where the models confidently predict ‘no’ ($\mathcal{P}_{eval}(4)$, shown in red). At the final layer in TS2.3 (e), the average distance between all the formats is 1.2, and the average cosine similarity is 0.998, despite the model having high confidence that the misclassified format is ‘no’, and high confidence the others are ‘yes’. However, while the format trajectories look closer at the final layer in TS2.4 (f), the average distance is 8.778, and the average cosine similarity is 0.769. This plot helps us visualise that despite having vastly different representational consistency metrics, they both achieve the same answer consistency. Additionally, as seen in (e) compared to (f), even when the representational consistency metrics suggest the model should be making more similar predictions, we still see confident misclassifications.

We see the paraphrase format represented by the red line misclassified not only in (e) and (f), but across all the models in TS2. This is $\mathcal{P}_{eval}(4)$, the 4th unseen format in $\mathcal{P}_{eval}$:

“Judging from ‘{input}’, is it correct to link the occurrence of ‘{effect}’ back to ‘{cause}’?”

Example 2

Input: “The expedition reached the Gulf of Saint Lawrence without incident, but foggy conditions, tricky currents, and strong winds combined to drive the fleet toward the north shore of the river near a place now called Pointe-aux-Anglais, where the ships were wrecked.”

Cause: “drive”

Effect: “wrecked”

Label: yes

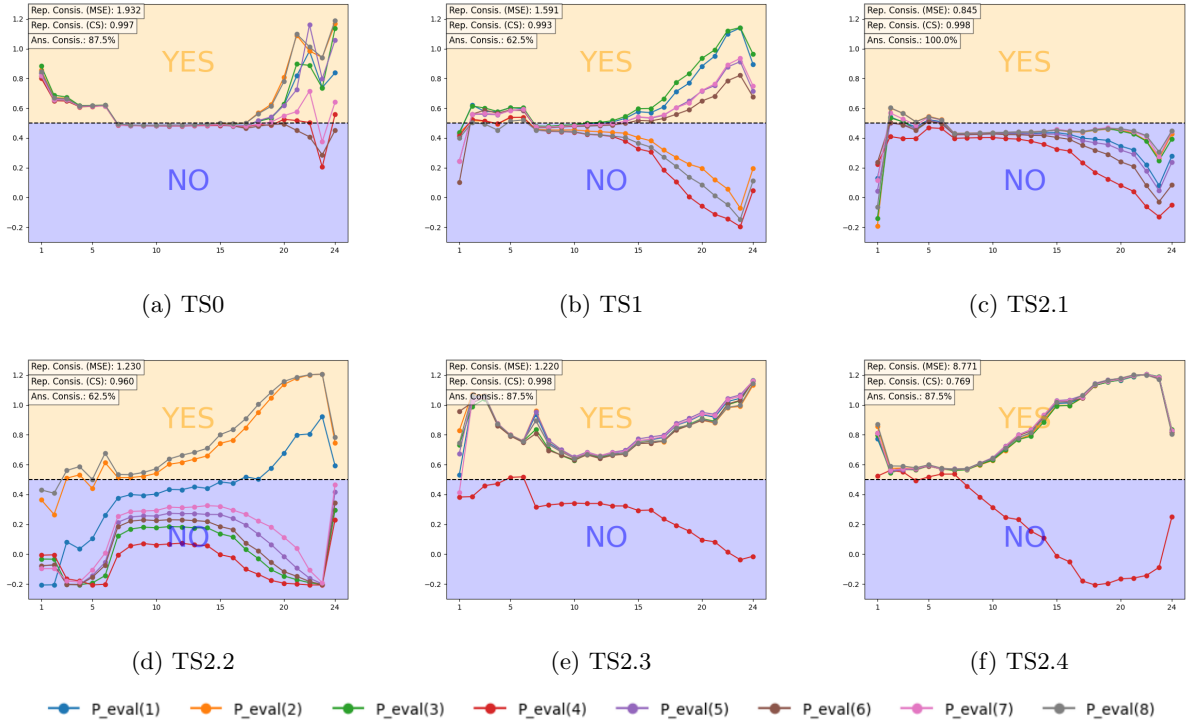


Figure 11: Decoder trajectory plots of the different training stages for Example 2. Each trajectory corresponds to a paraphrase format in $\mathcal{P}_{eval}$. Decoder layer number is represented on the x-axis. Normalised difference is represented on the y-axis.

While we observe similar plots for TS0 and TS1 (a, b) compared to the first example, the plots look rather different for the TS2 models. There is more variation in the trajectory of formats in TS2.1 and TS2.2. Especially in TS2.2, formats are mapped very differently in layers 1-23, with some aligning very strongly with the ‘yes’ logit, and others doing the same for the ‘no’ logit. Ultimately, this shows that even when trained with paraphrases, models can still be sensitive to the phrasing of the instruction. Additionally, we observe $\mathcal{P}_{eval}(4)$ being consistently misclassified again, as seen in TS2.3 and TS2.4 (e, f).

The difference in format mappings between the two examples, especially in TS2.1 and TS2.2, shows that the content tokens in the instruction can also significantly influence the trajectory of a format. If the content tokens had no effect on the consistency, we should expect to see similar trajectories through the decoder to those in Example 1, since the content is the only difference. However, by changing the content tokens, we observe significant variation in trajectories in (c) and (d). This implies that the content tokens have an affect on the difficulty of a given instruction format; a format may be benign for one input but adversarial for another. We leave determining this relationship between the content and format tokens, and how content tokens impact the difficulty of formats, for future work.

We include additional examples, and their associated decoder path plots in Appendix B.

4.2.3 Investigation into Evaluation Paraphrase 4

As seen in Figure 10 and Figure 11, $\mathcal{P}_{eval}(4)$ has a different trajectory through the decoder than the other formats, frequently being classified differently. We first seek to understand the effect of $\mathcal{P}_{eval}(4)$ on the consistency metrics. We evaluate each of the models on a subset of 1024 examples from the evaluation set, augmented with all evaluation paraphrases, except $\mathcal{P}_{eval}(4)$.

Training Regime	Avg. Norm ($\downarrow$)	Avg. Cos. Sim. ($\uparrow$)	Ans. Consistency ($\uparrow$)
TS0	2.233	0.9955	86.0%
TS1	0.998	0.9976	85.3%
TS2.1 (CE)	4.507	0.9977	97.4%
TS2.2 (CE-MSE)	0.483	0.9933	97.1%
TS2.3 (CE-CS)	0.792	0.9991	97.5%
TS2.4 (CE-CT)	1.571	0.9812	97.4%
TS2.4.1 (CE-CT)	0.688	0.9969	98.0%
TS2.4.2 (CE-CT)	0.840	0.9984	96.4%

Table 5: Consistency scores achieved by the training stages on a random subset of 1024 unique examples, augmented with seven of the eight evaluation paraphrase formats, excluding $\mathcal{P}_{eval}(4)$

As observed in Table 5, the removal of this paraphrase results in a significant increase in performance, with the highest performing method, TS2.4.1, now achieving an answer consistency of 98.0%, where the previous highest scoring methods had achieved 89.7%. We observe a decrease in average distance between representations for all the models in TS2. Likewise, there is also an increase in average cosine similarity between representations. This shows that $\mathcal{P}_{eval}(4)$ was having a significant impact on the results achieved by the TS2 models, skewing the results. The removal of the paraphrase format did not have as much impact on the performance of TS0 and TS1: not affecting the answer consistency of TS0, but moderately increasing that of TS1. This suggests that the TS2 models have somewhat overfit to the training paraphrase formats, as the impact of this particular format becomes more pronounced after training on the seen paraphrase formats $\mathcal{P}_{train}$.

To establish what about the format causes the model to classify it differently, we also experiment by making small changes to the format, and re-evaluating the models. The aim is to recover the consistency scores, with the inclusion of the adjusted $\mathcal{P}_{eval}(4)$, to be similar to those achieved in Table 5. We find that making the following change allows this format to be classified with the other formats in $\mathcal{P}_{eval}$:

Original format: “Judging from ‘{input}’, is it correct to link the occurrence of ‘{effect}’ **back** to ‘{cause}’?”

Adjusted format: “Judging from ‘{input}’, is it correct to link the occurrence of ‘{effect}’ **due** to ‘{cause}’?”

By changing the word ‘back’ to ‘due’, we achieve the results shown in Table 6:

Training Regime	Avg. Norm ($\downarrow$)	Avg. Cos. Sim. ($\uparrow$)	Ans. Consistency ($\uparrow$)
TS0	2.199	0.9960	86.2%
TS1	1.019	0.9977	82.9%
TS2.1 (CE)	4.408	0.9979	97.4%
TS2.2 (CE-MSE)	0.482	0.9935	96.9%
TS2.3 (CE-CS)	0.793	0.9993	97.4%
TS2.4 (CE-CT)	1.595	0.9809	97.3%
TS2.4.1 (CE-CT)	0.692	0.9968	97.9%
TS2.4.2 (CE-CT)	0.828	0.9986	96.4%

Table 6: Results achieved by the training stages on the same subset of unique examples as used for Table 5, where each example is augmented with all the formats in $\mathcal{P}_{eval}$, and where $\mathcal{P}_{eval}(4)$ is replaced with the adjusted format defined above.

We find that the inclusion of the word ‘due’ makes all the difference; empirically, we found that omitting it caused the results to revert back similar to those observed in Table 5. We also found this was seemingly the only word causing this difference in classification, changing other words had little effect on the results. What this shows is that while training on paraphrases does improve consistency for many paraphrase formats, it can still produce models which are sometimes vulnerable to small changes in exact wording. These ‘cliff edges’ in performance from minor wording changes suggest the model has overfit to the seen paraphrase formats $\mathcal{P}_{train}$, evidenced by the continued sensitivity. We believe the model is interpolating new formats, since some formats have almost identical trajectories through the decoder, but has not generalised due to sudden failure from minor changes.

4.2.4 Why Does Representational Consistency Not Affect Answer Consistency?

Here, we hypothesise why these two measures of consistency are not linked. On a random subset of 1024 examples from the evaluation set $\mathcal{V}$, phrased using every format in $\mathcal{P}_{eval}$ and $\mathcal{P}_{standard}$, all of the TS2 models answered either ‘yes’ or ‘no’ in 100% of cases. The model essentially maps a vector $\in \mathbb{R}^{1024}$ to a vector $\in \mathbb{R}^2$, since the output is one of two tokens. We hypothesise that in the 1022 directions orthogonal to this 2D subspace, the vector location has no effect on its mapping to this subspace, and doesn’t affect the output answer. Using this, we show how vector locations can vary greatly, producing seemingly poor representational consistency metrics, but not affecting answer consistency.

A Simple Example

We use a simple example to demonstrate this. Take two sets, each consisting of two $\mathbb{R}^2$ vectors $\{v_{11}, v_{12}\}$ and $\{v_{21}, v_{22}\}$, and an output matrix $M \in \mathbb{R}^{2 \times 2}$. The product of M and a vector v will be a scalar, which for the purpose of the example corresponds to ‘yes’ if positive, and ‘no’ if negative. We define these as follows:

$$v_{11} = \begin{pmatrix} 0.01 \\ 1 \end{pmatrix}, \quad v_{12} = \begin{pmatrix} -0.01 \\ 1 \end{pmatrix},$$

$$v_{21} = \begin{pmatrix} 0.01 \\ 10 \end{pmatrix}, \quad v_{22} = \begin{pmatrix} 0.01 \\ -10 \end{pmatrix},$$

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

We find the representational consistency between these two sets by taking the mean of the set, and applying the representational consistency metrics. To find the answer consistency, we find the product Mv . We show these results in Table 7.

Vector Set	Avg. Norm ($\downarrow$)	Avg. Cos. Sim. ($\uparrow$)	Ans. Consistency ($\uparrow$)
v_1	0.01	0.99995	50.0%
v_2	10	0.001	100.0%

Table 7: The representational and answer consistency of the vector sets $v_1 = \{v_{11}, v_{12}\}$ and $v_2 = \{v_{21}, v_{22}\}$, using the output matrix M

Since the scalar output is solely determined by the first dimension of the vectors, we can clearly see how vectors with high representational consistency can achieve poor answer consistency, and vice versa. We link this to the model representations, as the answer represented by the $\mathbb{R}^{1024}$ representations is determined by two dimensions, since we always output either ‘yes’ or ‘no’. Therefore, we show how a vector set can have poor representational consistency, but high answer consistency and vice versa, since many of the dimensions which impact the representational consistency do not impact the answer consistency.

We also use this to show why optimising for representational consistency at the final layer does not benefit answer consistency. Representational consistency objectives will aim to improve the consistency of the representations across all dimensions. As discussed, two dimensions determine the output of a 1024 dimension vector, while 1022 of the dimensions have no effect on the output. However, these training objectives give all dimensions equal weighting. Therefore, much of the optimisation has no impact on the output. Additionally, the objective may choose to sacrifice consistency in the two dimensions which *do* determine the output, for consistency in dimensions which *don't*. Thereby sacrificing answer consistency in favour of representational consistency.

5 Future Work and Limitations

We conducted a comprehensive analysis using a small model on a single binary classification causal reasoning dataset. Future work could evaluate the extent to which our findings carry over to other IFT tasks and larger models, especially those which emergent abilities.

While the training pipeline we presented improved performance and consistency on a variety of paraphrase formats, we saw how some models in TS2 were still vulnerable to small changes in wording. This suggests that to some degree, the TS2 models were interpolating unseen paraphrase formats, rather than truly generalising. Future work could seek to understand why all of the TS2 models were vulnerable the the specific wording of $\mathcal{P}_{eval}(4)$, and why the effects of the wording were so extreme. Additionally, we observed how the difficulty of a paraphrase format was influenced by the content of the input. Future work could establish the relationship between the input content and the format it takes, and how formats can become more challenging for different inputs. We believe that analysis of the ways in which the decoder incorporates information from the encoder in the cross attention module could yield insights for both of these suggestions.

Perhaps more importantly, future work could further analyse our hypothesis regarding the relationship between final layer representational consistency and answer consistency. We found that improving representational consistency at the final layer did not correlate with better answer consistency, and hypothesised why this may be the case in Section 4.2.4. Better understanding of this relationship could allow for the development of training objectives which improves answer consistency by improving representational consistency in different ways to what we explored.

6 Conclusion

In this dissertation, we conducted a comprehensive evaluation of the impact of instruction format on the performance and consistency of FLAN-T5-Large on a causal reasoning task. We fine-tuned the model using twenty different instruction formats, and found this significantly improved performance on an evaluation set where each example uses one of eight unseen instruction formats. We also found this significantly increased robustness to the unseen formats, as we observe the model outputs become more consistent under the formats. We also evaluated the impact of a number of different training objectives which encourage more consistent final layer representations, finding that while representational consistency improved, this did not lead to improved answer consistency. We then presented a hypothesis explaining why this might be the case. Additionally, we showed how even when representational consistency was high, answer consistency can still be low, using a simple example, and our *decoder trajectory plot* to illustrate this. Ultimately, we highlight the importance of training not just on a variety of examples, but also a variety of instruction formats, to achieve robust models which perform better in real-world presentations of downstream tasks.

Word Count: 10,893

References

- Agrawal, A., Alazraki, L., Honarvar, S., Mensink, T., and Rei, M., 2025. Enhancing LLM robustness to perturbed instructions: an empirical study. *Iclr 2025 workshop on building trust in language models and applications* [Online]. Available from: <https://openreview.net/forum?id=abl1mCsDp8>.
- Alzantot, M., Sharma, Y., Elgohary, A., Ho, B.-J., Srivastava, M., and Chang, K.-W., 2018. Generating natural language adversarial examples. In: E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, eds. *Proceedings of the 2018 conference on empirical methods in natural language processing* [Online]. Brussels, Belgium: Association for Computational Linguistics, pp.2890–2896. Available from: <https://doi.org/10.18653/v1/D18-1316>.
- Biggio, B., Corona, I., Maiorca, D., Nelson, B., Šrncić, N., Laskov, P., Giacinto, G., and Roli, F., 2013. Evasion attacks against machine learning at test time. In: *Advanced information systems engineering* [Online]. Springer Berlin Heidelberg, pp.387–402. Available from: https://doi.org/10.1007/978-3-642-40994-3_25.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D., 2020. Language models are few-shot learners. In: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, eds. *Advances in neural information processing systems* [Online]. Vol. 33. Curran Associates, Inc., pp.1877–1901. Available from: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y.T., Li, Y., Lundberg, S., Nori, H., Palangi, H., Ribeiro, M.T., and Zhang, Y., 2023. *Sparks of artificial general intelligence: early experiments with gpt-4* [Online]. arXiv: 2303.12712 [cs.CL]. Available from: <https://arxiv.org/abs/2303.12712>.
- Caselli, T. and Vossen, P., 2017. The event StoryLine corpus: a new benchmark for causal and temporal relation extraction. In: T. Caselli, B. Miller, M. van Erp, P. Vossen, M. Palmer, E. Hovy, T. Mitamura, and D. Caswell, eds. *Proceedings of the events and stories in the news workshop* [Online]. Vancouver, Canada: Association for Computational Linguistics, pp.77–86. Available from: <https://doi.org/10.18653/v1/W17-2711>.
- Cer, D., Yang, Y., Kong, S.-y., Hua, N., Limtiaco, N., John, R.S., Constant, N., Guajardo-Cespedes, M., Yuan, S., Tar, C., Sung, Y.-H., Strophe, B., and Kurzweil, R., 2018. *Universal sentence encoder* [Online]. arXiv: 1803.11175 [cs.CL]. Available from: <https://arxiv.org/abs/1803.11175>.
- Chung, H.W., Hou, L., Longpre, S., Zoph, B., Tai, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., Webson, A., Gu, S.S., Dai, Z., Suzgun, M., Chen, X., Chowdhery, A., Castro-Ros, A., Pellat, M., Robinson, K., Valter, D., Narang, S., Mishra, G., Yu, A., Zhao, V., Huang, Y., Dai, A., Yu, H., Petrov, S., Chi, E.H., Dean, J., Devlin, J., Roberts, A., Zhou, D., Le, Q.V., and Wei, J., 2024. Scaling instruction-finetuned language models. *J. mach. learn. res.*, 25(1).
- Creswell, A., Shanahan, M., and Higgins, I., 2023. Selection-inference: exploiting large language models for interpretable logical reasoning. *The eleventh international conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=3Pf3Wg6o-A4>.
- Dong, L., Mallinson, J., Reddy, S., and Lapata, M., 2017. Learning to paraphrase for question answering. In: M. Palmer, R. Hwa, and S. Riedel, eds. *Proceedings of the 2017 conference on empirical methods in natural language processing* [Online]. Copenhagen, Denmark: Association for Computational Linguistics, pp.875–886. Available from: <https://doi.org/10.18653/v1/D17-1091>.

- Ebrahimi, J., Lowd, D., and Dou, D., 2018. On adversarial examples for character-level neural machine translation. In: E.M. Bender, L. Derczynski, and P. Isabelle, eds. *Proceedings of the 27th international conference on computational linguistics* [Online]. Santa Fe, New Mexico, USA: Association for Computational Linguistics, pp.653–663. Available from: <https://aclanthology.org/C18-1055/>.
- Ebrahimi, J., Rao, A., Lowd, D., and Dou, D., 2018. HotFlip: white-box adversarial examples for text classification. In: I. Gurevych and Y. Miyao, eds. *Proceedings of the 56th annual meeting of the association for computational linguistics (volume 2: short papers)* [Online]. Melbourne, Australia: Association for Computational Linguistics, pp.31–36. Available from: <https://doi.org/10.18653/v1/P18-2006>.
- Fu Yao; Peng, H. and Khot, T., 2022. How does gpt obtain its ability? tracing emergent abilities of language models to their sources. *Yao fu’s notion* [Online]. Available from: <https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fcf74f30a1ab9e3e36fa1dc1>.
- Gao, J., Lanchantin, J., Soffa, M.L., and Qi, Y., 2018. Black-box generation of adversarial text sequences to evade deep learning classifiers. *2018 ieee security and privacy workshops (spw)* [Online], pp.50–56. Available from: <https://doi.org/10.1109/SPW.2018.00016>.
- Gao, J., Ding, X., Qin, B., and Liu, T., 2023. Is ChatGPT a good causal reasoner? a comprehensive evaluation. In: H. Bouamor, J. Pino, and K. Bali, eds. *Findings of the association for computational linguistics: emnlp 2023* [Online]. Singapore: Association for Computational Linguistics, pp.11111–11126. Available from: <https://doi.org/10.18653/v1/2023.findings-emnlp.743>.
- Goodfellow, I.J., Shlens, J., and Szegedy, C., 2015. *Explaining and harnessing adversarial examples* [Online]. arXiv: 1412.6572 [stat.ML]. Available from: <https://arxiv.org/abs/1412.6572>.
- Gu, J., Zhao, H., Xu, H., Nie, L., Mei, H., and Yin, W., 2023. Robustness of learning from task instructions. In: A. Rogers, J. Boyd-Graber, and N. Okazaki, eds. *Findings of the association for computational linguistics: acl 2023* [Online]. Toronto, Canada: Association for Computational Linguistics, pp.13935–13948. Available from: <https://doi.org/10.18653/v1/2023.findings-acl.875>.
- Gunel, B., Du, J., Conneau, A., and Stoyanov, V., 2021. Supervised contrastive learning for pre-trained language model fine-tuning. *International conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=cu7IUi0hujH>.
- Heigold, G., Varanasi, S., Neumann, G., and Genabith, J. van, 2018. How robust are character-based word embeddings in tagging and MT against word scrambling or random noise? In: C. Cherry and G. Neubig, eds. *Proceedings of the 13th conference of the association for machine translation in the Americas (volume 1: research track)* [Online]. Boston, MA: Association for Machine Translation in the Americas, pp.68–80. Available from: <https://aclanthology.org/W18-1807/>.
- Hobbhahn, M., Lieberum, T., and Seiler, D., 2022. Investigating causal understanding in LLMs. *Neurips 2022 workshop on causality for real-world impact* [Online]. Available from: <https://openreview.net/forum?id=st6jtGdW8Ke>.
- Huang, J. and Chang, K.C.-C., 2023. Towards reasoning in large language models: a survey. In: A. Rogers, J. Boyd-Graber, and N. Okazaki, eds. *Findings of the association for computational linguistics: acl 2023* [Online]. Toronto, Canada: Association for Computational Linguistics, pp.1049–1065. Available from: <https://doi.org/10.18653/v1/2023.findings-acl.67>.
- Ilyas, A., Santurkar, S., Tsipras, D., Engstrom, L., Tran, B., and Madry, A., 2019. Adversarial examples are not bugs, they are features. In: H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, eds. *Advances in neural information processing systems* [Online]. Vol. 32. Curran Associates, Inc. Available from: https://proceedings.neurips.cc/paper_files/paper/2019/file/e2c420d928d4bf8ce0ff2ec19b371514-Paper.pdf.

- Iyyer, M., Wieting, J., Gimpel, K., and Zettlemoyer, L., 2018. Adversarial example generation with syntactically controlled paraphrase networks. In: M. Walker, H. Ji, and A. Stent, eds. *Proceedings of the 2018 conference of the north American chapter of the association for computational linguistics: human language technologies, volume 1 (long papers)* [Online]. New Orleans, Louisiana: Association for Computational Linguistics, pp.1875–1885. Available from: <https://doi.org/10.18653/v1/N18-1170>.
- Jin, D., Jin, Z., Zhou, J.T., and Szolovits, P., 2020. *Is bert really robust? a strong baseline for natural language attack on text classification and entailment* [Online]. arXiv: 1907.11932 [cs.CL]. Available from: <https://arxiv.org/abs/1907.11932>.
- Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C., and Krishnan, D., 2020. Supervised contrastive learning. In: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, eds. *Advances in neural information processing systems* [Online]. Vol. 33. Curran Associates, Inc., pp.18661–18673. Available from: https://proceedings.neurips.cc/paper_files/paper/2020/file/d89a66c7c80a29b1bdbab0f2a1a94af8-Paper.pdf.
- Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., and Iwasawa, Y., 2022. Large language models are zero-shot reasoners. *Proceedings of the 36th international conference on neural information processing systems, Nips '22*. New Orleans, LA, USA: Curran Associates Inc.
- Krizhevsky, A., Sutskever, I., and Hinton, G.E., 2017. Imagenet classification with deep convolutional neural networks. *Commun. acm* [Online], 60(6), pp.84–90. Available from: <https://doi.org/10.1145/3065386>.
- Lampinen, A., Dasgupta, I., Chan, S., Mathewson, K., Tessler, M., Creswell, A., McClelland, J., Wang, J., and Hill, F., 2022. Can language models learn from explanations in context? In: Y. Goldberg, Z. Kozareva, and Y. Zhang, eds. *Findings of the association for computational linguistics: emnlp 2022* [Online]. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp.537–563. Available from: <https://doi.org/10.18653/v1/2022.findings-emnlp.38>.
- Le, Q.V., Ranzato, M., Monga, R., Devin, M., Chen, K., Corrado, G.S., Dean, J., and Ng, A.Y., 2012. *Building high-level features using large scale unsupervised learning* [Online]. arXiv: 1112.6209 [cs.LG]. Available from: <https://arxiv.org/abs/1112.6209>.
- LeCun, Y., Cortes, C., and Burges, C., 2010. Mnist handwritten digit database. *Att labs [online]*. available: <http://yann.lecun.com/exdb/mnist>, 2.
- Li, J., Ji, S., Du, T., Li, B., and Wang, T., 2019. Textbugger: generating adversarial text against real-world applications. *Proceedings 2019 network and distributed system security symposium* [Online], Ndss 2019. Internet Society. Available from: <https://doi.org/10.14722/ndss.2019.23138>.
- Li, L., Ma, R., Guo, Q., Xue, X., and Qiu, X., 2020. BERT-ATTACK: adversarial attack against BERT using BERT. In: B. Webber, T. Cohn, Y. He, and Y. Liu, eds. *Proceedings of the 2020 conference on empirical methods in natural language processing (emnlp)* [Online]. Online: Association for Computational Linguistics, pp.6193–6202. Available from: <https://doi.org/10.18653/v1/2020.emnlp-main.500>.
- Li, Z., Peng, B., He, P., and Yan, X., 2024. Evaluating the instruction-following robustness of large language models to prompt injection. In: Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, eds. *Proceedings of the 2024 conference on empirical methods in natural language processing* [Online]. Miami, Florida, USA: Association for Computational Linguistics, pp.557–568. Available from: <https://doi.org/10.18653/v1/2024.emnlp-main.33>.
- Long, S., Schuster, T., and Piché, A., 2024. *Can large language models build causal graphs?* [Online]. arXiv: 2303.05279 [cs.CL]. Available from: <https://arxiv.org/abs/2303.05279>.

- Madaan, A., Zhou, S., Alon, U., Yang, Y., and Neubig, G., 2022. Language models of code are few-shot commonsense learners. In: Y. Goldberg, Z. Kozareva, and Y. Zhang, eds. *Proceedings of the 2022 conference on empirical methods in natural language processing* [Online]. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp.1384–1403. Available from: <https://doi.org/10.18653/v1/2022.emnlp-main.90>.
- Morris, J., Lifland, E., Lanchantin, J., Ji, Y., and Qi, Y., 2020. Reevaluating adversarial examples in natural language. In: T. Cohn, Y. He, and Y. Liu, eds. *Findings of the association for computational linguistics: emnlp 2020* [Online]. Online: Association for Computational Linguistics, pp.3829–3839. Available from: <https://doi.org/10.18653/v1/2020.findings-emnlp.341>.
- Musgrave, K., Belongie, S., and Lim, S.-N., 2020. *Pytorch metric learning* [Online]. arXiv: 2008.09164 [cs.CV]. Available from: <https://arxiv.org/abs/2008.09164>.
- Papernot, N., McDaniel, P., Swami, A., and Harang, R., 2016. Crafting adversarial input sequences for recurrent neural networks. *Milcom 2016 - 2016 ieee military communications conference* [Online]. Baltimore, MD, USA: IEEE Press, pp.49–54. Available from: <https://doi.org/10.1109/MILCOM.2016.7795300>.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I., 2019. Language models are unsupervised multitask learners [Online]. Available from: <https://api.semanticscholar.org/CorpusID:160025533>.
- Rae, J.W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., Rutherford, E., Hennigan, T., Menick, J., Cassirer, A., Powell, R., Driessche, G. van den, Hendricks, L.A., Rauh, M., Huang, P.-S., Glaese, A., Welbl, J., Dhathathri, S., Huang, S., Uesato, J., Mellor, J., Higgins, I., Creswell, A., McAleese, N., Wu, A., Elsen, E., Jayakumar, S., Buchatskaya, E., Budden, D., Sutherland, E., Simonyan, K., Paganini, M., Sifre, L., Martens, L., Li, X.L., Kuncoro, A., Nematzadeh, A., Gribovskaya, E., Donato, D., Lazaridou, A., Mensch, A., Lepiau, J.-B., Tsimpoukelli, M., Grigorev, N., Fritz, D., Sottiaux, T., Pajarskas, M., Pohlen, T., Gong, Z., Toyama, D., Masson d’Autume, C. de, Li, Y., Terzi, T., Mikulik, V., Babuschkin, I., Clark, A., Las Casas, D. de, Guy, A., Jones, C., Bradbury, J., Johnson, M., Hechtman, B., Weidinger, L., Gabriel, I., Isaac, W., Lockhart, E., Osindero, S., Rimell, L., Dyer, C., Vinyals, O., Ayoub, K., Stanway, J., Bennett, L., Hassabis, D., Kavukcuoglu, K., and Irving, G., 2022. *Scaling language models: methods, analysis & insights from training gopher* [Online]. arXiv: 2112.11446 [cs.CL]. Available from: <https://arxiv.org/abs/2112.11446>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. mach. learn. res.*, 21(1).
- Ren, S., Deng, Y., He, K., and Che, W., 2019. Generating natural language adversarial examples through probability weighted word saliency. In: A. Korhonen, D. Traum, and L. Màrquez, eds. *Proceedings of the 57th annual meeting of the association for computational linguistics* [Online]. Florence, Italy: Association for Computational Linguistics, pp.1085–1097. Available from: <https://doi.org/10.18653/v1/P19-1103>.
- Ruis, L., Khan, A., Biderman, S., Hooker, S., Rocktäschel, T., and Grefenstette, E., 2023. The goldilocks of pragmatic understanding: fine-tuning strategy matters for implicature resolution by llms. *Proceedings of the 37th international conference on neural information processing systems*, Nips ’23. New Orleans, LA, USA: Curran Associates Inc.
- Sakaguchi, K., Duh, K., Post, M., and Van Durme, B., 2017. Robust word recognition via semi-character recurrent neural network. Publisher Copyright: Copyright © 2017, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.; 31st AAAI Conference on Artificial Intelligence, AAAI 2017 ; Conference date: 04-02-2017 Through 10-02-2017, pp.3281–3287.

- Sanh, V., Webson, A., Raffel, C., Bach, S., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Raja, A., Dey, M., Bari, M.S., Xu, C., Thakker, U., Sharma, S.S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N., Datta, D., Chang, J., Jiang, M.T.-J., Wang, H., Manica, M., Shen, S., Yong, Z.X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Fevry, T., Fries, J.A., Teehan, R., Scao, T.L., Biderman, S., Gao, L., Wolf, T., and Rush, A.M., 2022. Multitask prompted training enables zero-shot task generalization. *International conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=9Vrb9D0WI4>.
- Sun, J., Shaib, C., and Wallace, B.C., 2024. Evaluating the zero-shot robustness of instruction-tuned language models. *The twelfth international conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=g9diuvxN6D>.
- Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R., 2014. *Intriguing properties of neural networks* [Online]. arXiv: 1312.6199 [cs.CV]. Available from: <https://arxiv.org/abs/1312.6199>.
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T.B., 2023. *Stanford alpaca: an instruction-following llama model*. https://github.com/tatsu-lab/stanford_alpaca. GitHub.
- Terven, J., Cordova-Esparza, D.M., Ramirez-Pedraza, A., Chavez-Urbiola, E.A., and Romero-Gonzalez, J.A., 2024. *Loss functions and metrics in deep learning* [Online]. arXiv: 2307.02694 [cs.LG]. Available from: <https://arxiv.org/abs/2307.02694>.
- Tsipras, D., Santurkar, S., Engstrom, L., Turner, A., and Madry, A., 2019. Robustness may be at odds with accuracy. *International conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=SyxAb30cY7>.
- Tu, R., Ma, C., and Zhang, C., 2023. *Causal-discovery performance of chatgpt in the context of neuropathic pain diagnosis* [Online]. arXiv: 2301.13819 [cs.CL]. Available from: <https://arxiv.org/abs/2301.13819>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I., 2017. Attention is all you need. In: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds. *Advances in neural information processing systems* [Online]. Vol. 30. Curran Associates, Inc. Available from: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Wang, B., Pei, H., Pan, B., Chen, Q., Wang, S., and Li, B., 2020a. T3: tree-autoencoder constrained adversarial text generation for targeted attack. In: B. Webber, T. Cohn, Y. He, and Y. Liu, eds. *Proceedings of the 2020 conference on empirical methods in natural language processing (emnlp)* [Online]. Online: Association for Computational Linguistics, pp.6134–6150. Available from: <https://doi.org/10.18653/v1/2020.emnlp-main.495>.
- Wang, L., Ghosh, D., Diaz, M.T.G., Farahat, A., Alam, M., Gupta, C., Chen, J., and Marathe, M., 2020b. Wisdom of the ensemble: improving consistency of deep learning models. *Proceedings of the 34th international conference on neural information processing systems*, Nips '20. Vancouver, BC, Canada: Curran Associates Inc.
- Wang, X., Chen, Y., Ding, N., Peng, H., Wang, Z., Lin, Y., Han, X., Hou, L., Li, J., Liu, Z., Li, P., and Zhou, J., 2022. MAVEN-ERE: a unified large-scale dataset for event coreference, temporal, causal, and subevent relation extraction. In: Y. Goldberg, Z. Kozareva, and Y. Zhang, eds. *Proceedings of the 2022 conference on empirical methods in natural language processing* [Online]. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, pp.926–941. Available from: <https://doi.org/10.18653/v1/2022.emnlp-main.60>.
- Wang, X., Wei, J., Schuurmans, D., Le, Q.V., Chi, E.H., Narang, S., Chowdhery, A., and Zhou, D., 2023. Self-consistency improves chain of thought reasoning in language models. *The eleventh international conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=1PL1NIMMrw>.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E.H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., and Fedus, W., 2022a. Emergent abilities of large language models. *Transactions on machine learning research* [Online]. Survey Certification. Available from: <https://openreview.net/forum?id=yzkSU5zdwD>.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., and Zhou, D., 2022b. Chain-of-thought prompting elicits reasoning in large language models. *Proceedings of the 36th international conference on neural information processing systems*, Nips '22. New Orleans, LA, USA: Curran Associates Inc.

Yan, L.T., Wang, F., Huang, J.Y., Zhou, W., Yin, F., Galstyan, A., Yin, W., and Chen, M., 2024. Contrastive instruction tuning. *Acl - findings*.

Zhang, Y., Wang, X., Xi, Z., Xia, H., Gui, T., Zhang, Q., and Huang, X., 2024. RoCoIns: enhancing robustness of large language models through code-style instructions. In: N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, eds. *Proceedings of the 2024 joint international conference on computational linguistics, language resources and evaluation (lrec-coling 2024)* [Online]. Torino, Italia: ELRA and ICCL, pp.14186–14203. Available from: <https://aclanthology.org/2024.lrec-main.1237/>.

Zou, J., Zhang, S., and Qiu, M., 2024. Adversarial attacks on large language models. *Knowledge science, engineering and management: 17th international conference, ksem 2024, birmingham, uk, august 16–18, 2024, proceedings, part iv* [Online]. Birmingham, United Kingdom: Springer-Verlag, pp.85–96. Available from: https://doi.org/10.1007/978-981-97-5501-1_7.

A Paraphrase Formats

A.1 Training Paraphrase Formats

1. "Considering the input: '{input}', can we conclude that '{cause}' is responsible for causing '{effect}'?"
2. "Given the input: '{input}', can '{cause}' be attributed to causing '{effect}'?"
3. "Is '{cause}' a necessary condition for '{effect}' to happen, given the following input: '{input}'?"
4. "Is '{effect}' a consequence of '{cause}', given the input '{input}'?"
5. "Does the input '{input}' indicate '{cause}' caused '{effect}' to happen?"
6. "Does the input support the claim '{cause}' caused '{effect}', input: '{input}'?"
7. "Consider this input: '{input}', is '{cause}' the reason '{effect}' happened?"
8. "Based on the input '{input}', is it reasonable to conclude '{cause}' lead to '{effect}' happening?"
9. "If '{input}' is true, does it follow that '{cause}' resulted in '{effect}'?"
10. "After examining the input '{input}', should we infer '{cause}' lead to '{effect}'?"
11. "From the following input: '{input}', can we reasonably argue '{cause}' triggered the occurrence of '{effect}'?"
12. "Considering this input: '{input}', does '{effect}' explicitly arise due to '{cause}'?"
13. "Given this input: '{input}', is it accurate to say '{cause}' produced '{effect}'?"
14. "If we accept the following information: '{input}', does this imply that '{effect}' happened specifically because '{cause}' occurred?"
15. "Can we infer from the following input: '{input}' that '{cause}' was the primary factor leading to '{effect}'?"
16. "Given the scenario '{input}', did '{effect}' occur as a direct consequence of '{cause}'?"
17. "If we rely on this input: '{input}', is '{cause}' what specifically caused '{effect}' to occur?"
18. "According to this: '{input}', was '{cause}' instrumental in bringing about '{effect}'?"
19. "Analyzing this information: '{input}', can we determine whether '{effect}' was initiated by '{cause}'?"
20. "Is '{cause}' explicitly identified as the trigger for '{effect}', according to the input: '{input}'?"

A.2 Evaluation Paraphrase Formats

1. "Taking the statement '{input}' into account, is '{cause}' the event that made '{effect}' occur?"
2. "If '{input}' holds true, can we definitively state that '{effect}' was brought about by '{cause}'?"
3. "Does '{cause}' directly explain why '{effect}' took place, based on the information: '{input}'?"
4. "Judging from '{input}', is it correct to link the occurrence of '{effect}' back to '{cause}'?"
5. "If we consider '{input}', does it logically suggest that '{cause}' is behind the event '{effect}'?"
6. "Based on your understanding of '{input}', would you say that '{cause}' instigated '{effect}'?"
7. "Reviewing '{input}', is it valid to claim '{cause}' was the catalyst for '{effect}'?"
8. "Does the provided scenario '{input}' imply that the event '{effect}' was the outcome caused by '{cause}'?"

B Additional Decoder Trajectory Plots

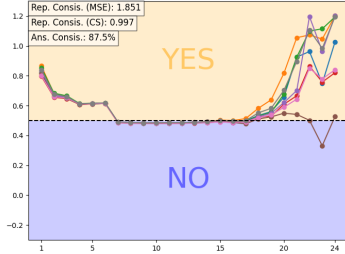
B.1 Additional Examples from Training Set $\mathcal{T}$

Input: The 1996 Mount Everest disaster occurred on 10–11 May 1996, when eight people caught in a blizzard died on Mount Everest during attempts to descend from the summit.

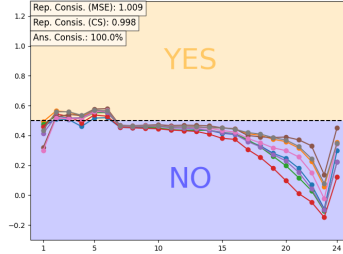
Cause: disaster

Effect: died

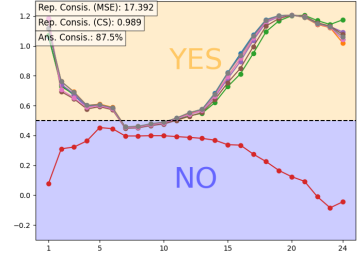
Label: yes



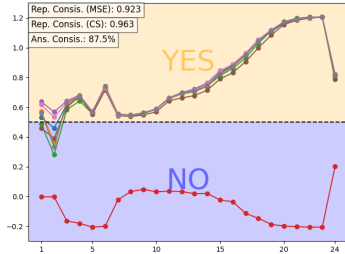
(a) TS0



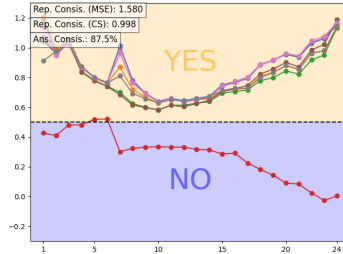
(b) TS1



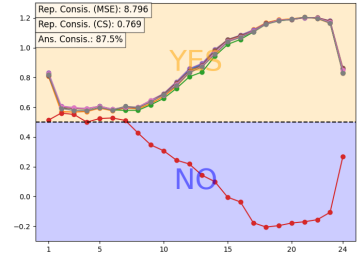
(c) TS2.1



(d) TS2.2



(e) TS2.3



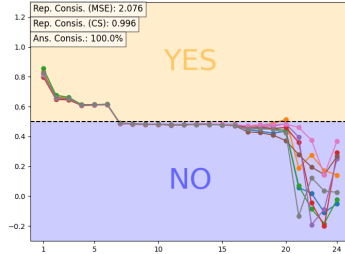
(f) TS2.4

Input: The route is now flown by a Boeing 747-400 and Airbus A330-300. Flight 642 was one of only two hull losses of MD-11s with passenger configuration, the other being Swissair Flight 111, which crashed in 1998 with 229 fatalities.

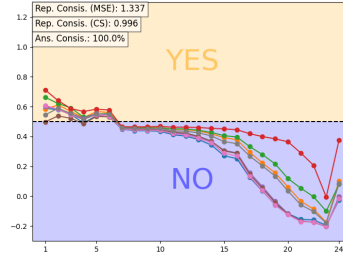
Cause: flown

Effect: crashed

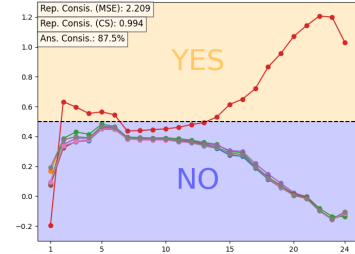
Label: no



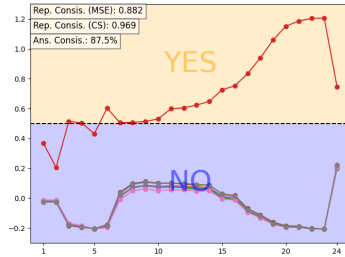
(a) TS0



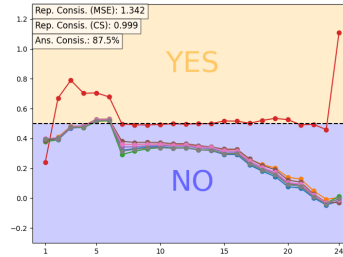
(b) TS1



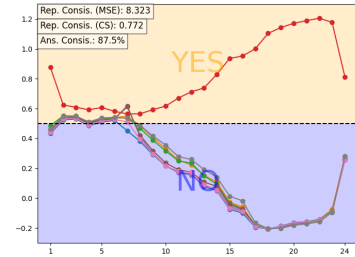
(c) TS2.1



(d) TS2.2



(e) TS2.3



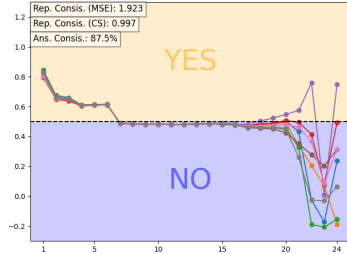
(f) TS2.4

Input: The explosion killed eight people and injured at least 209 people, twelve severely. The second attack occurred less than two hours later at a summer camp on the island of Utøya in Tyrifjorden, Buskerud.

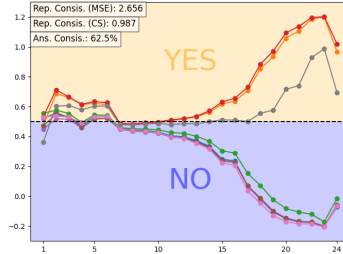
Cause: injured

Effect: attack

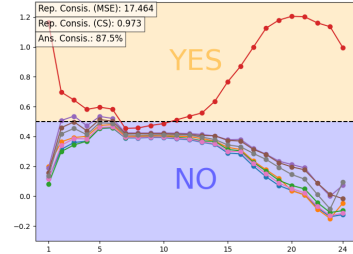
Label: no



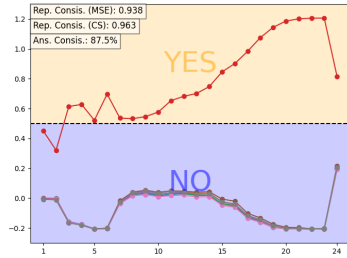
(a) TS0



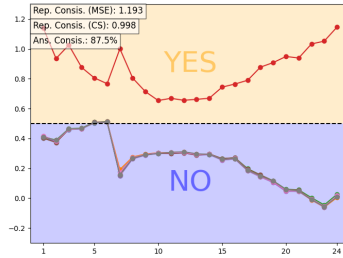
(b) TS1



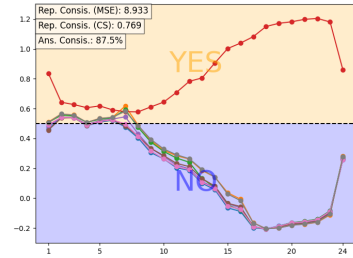
(c) TS2.1



(d) TS2.2



(e) TS2.3



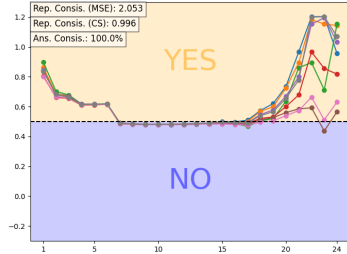
(f) TS2.4

Input: The initial judicial inquiry found "Mont-Blanc" to have been responsible for the disaster, but a later appeal determined that both vessels were to blame.

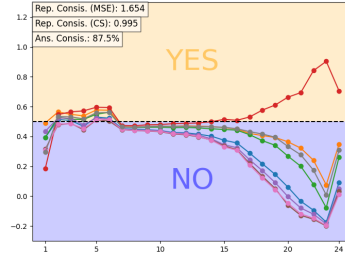
Cause: determined

Effect: appeal

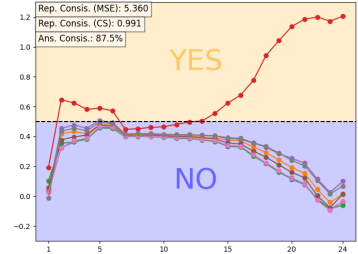
Label: no



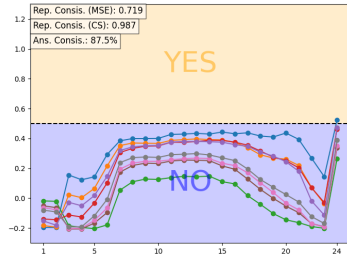
(a) TS0



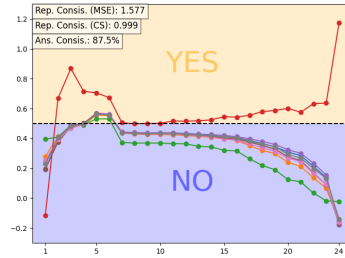
(b) TS1



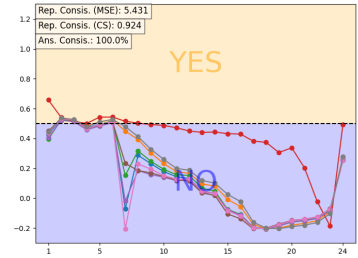
(c) TS2.1



(d) TS2.2



(e) TS2.3



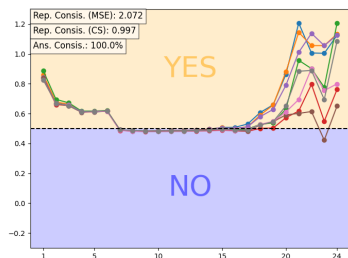
(f) TS2.4

Input: Previously unreleased tracks from her then-upcoming second album were previewed during the tour.
The singer described the tour as an exclusive event for fans to showcase the album's musical direction.

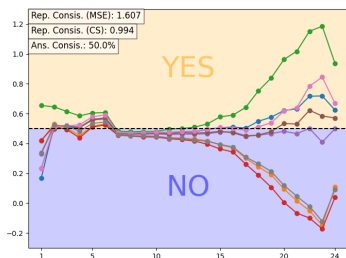
Cause: tour

Effect: described

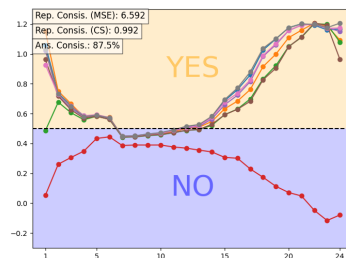
Label: yes



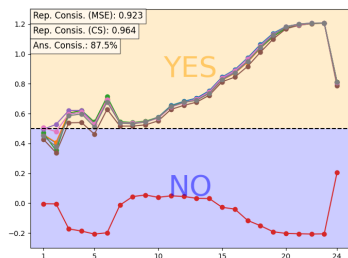
(a) TS0



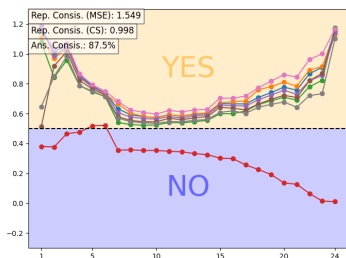
(b) TS1



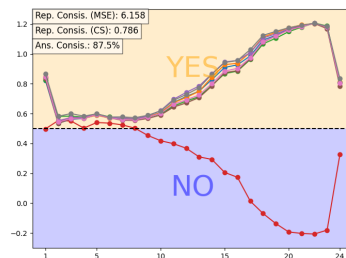
(c) TS2.1



(d) TS2.2



(e) TS2.3



(f) TS2.4

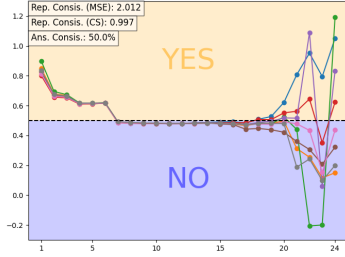
B.2 Additional Examples from Evaluation Set $\mathcal{V}$

Input: The 2003 Youth Asia Cup was an international under-19 cricket tournament held in Karachi, Pakistan, from 14 to 27 July 2003. The fifth ACC under-19 tournament to be held, it was originally scheduled to be held in Singapore, but was moved to Karachi as a precaution against the SARS outbreak.

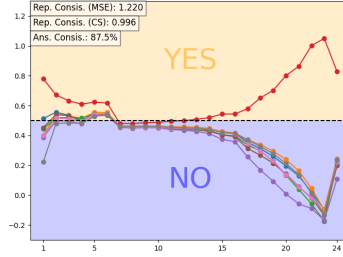
Cause: held

Effect: moved

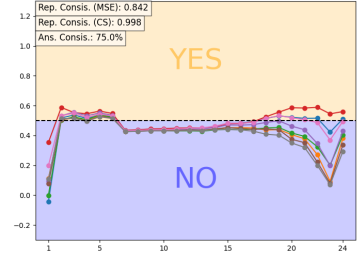
Label: no



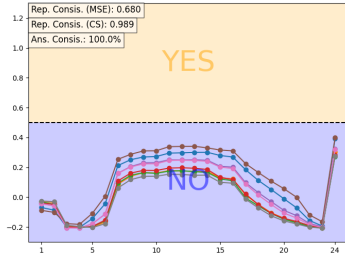
(a) TS0



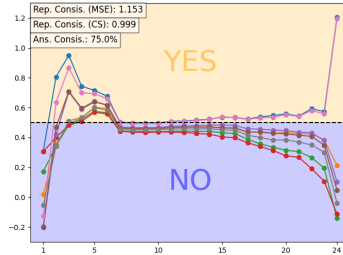
(b) TS1



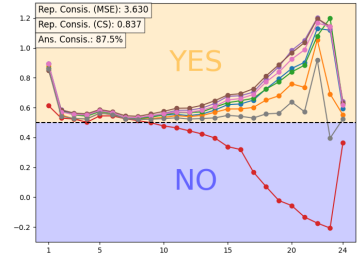
(c) TS2.1



(d) TS2.2



(e) TS2.3



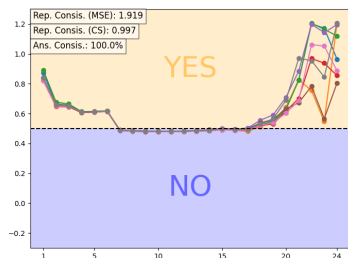
(f) TS2.4

Input: In reaction to a state visit by the Japanese Prime Minister, Kakuei Tanaka, students held a demonstration protesting corruption, high prices, and inequality in foreign investments. After provocation by suspected agent provocateurs, the demonstrations became riots, which eventually turned into a pogrom.

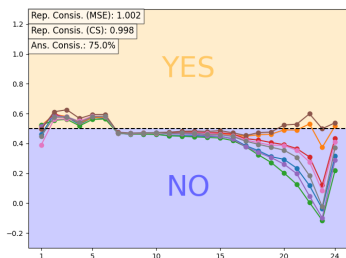
Cause: protesting

Effect: became

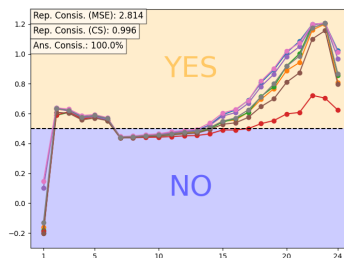
Label: yes



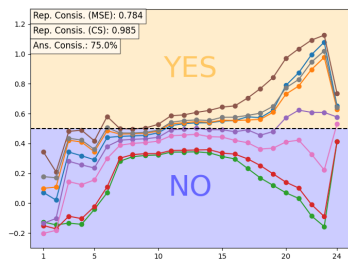
(a) TS0



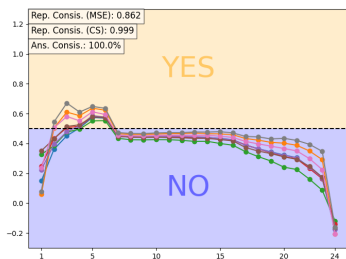
(b) TS1



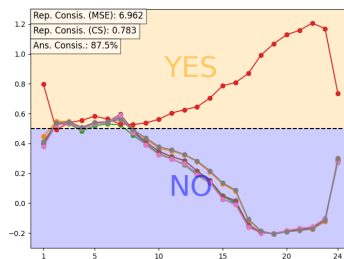
(c) TS2.1



(d) TS2.2



(e) TS2.3



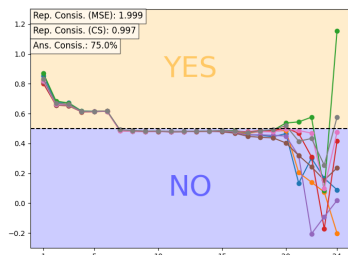
(f) TS2.4

Input: A British squadron had been blockading the French island of Isle de France (now Mauritius) since early in the year, and early on 5 May discovered two ships approaching their position. As the strange vessels came closer, they were recognised as the French frigate "Duguay Trouin", which had been captured from the East India Company the year before, and a small brig.

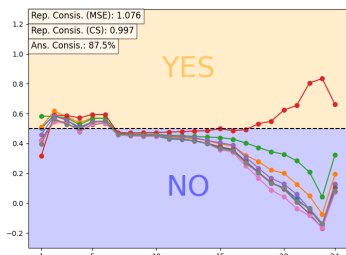
Cause: discovered

Effect: captured

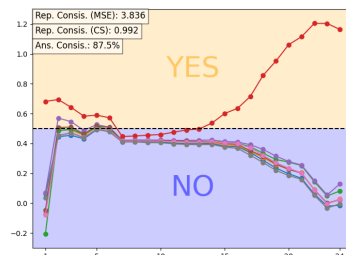
Label: no



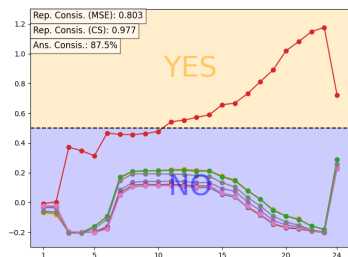
(a) TS0



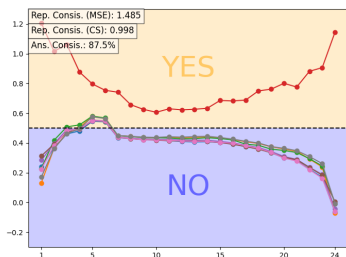
(b) TS1



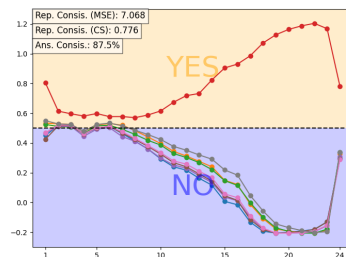
(c) TS2.1



(d) TS2.2



(e) TS2.3



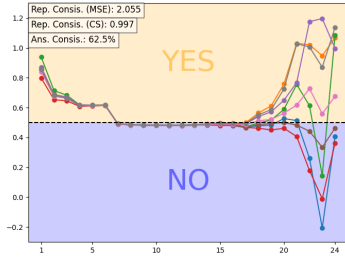
(f) TS2.4

Input: Gates found the front door to his home jammed shut and, with the help of his driver, tried to force it open. A local witness reported their activity to the police as a potential burglary in progress.

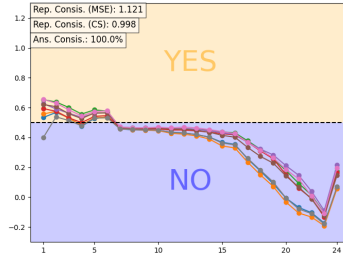
Cause: help

Effect: reported

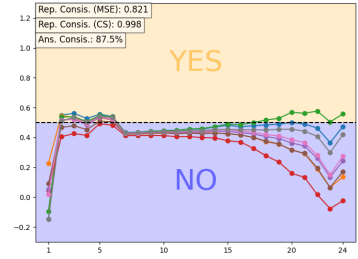
Label: yes



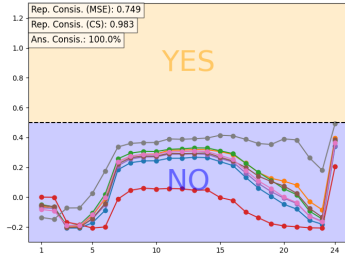
(a) TS0



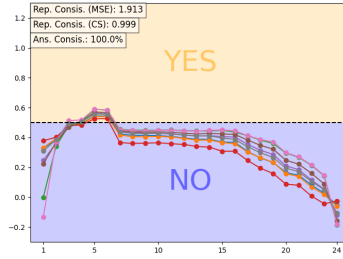
(b) TS1



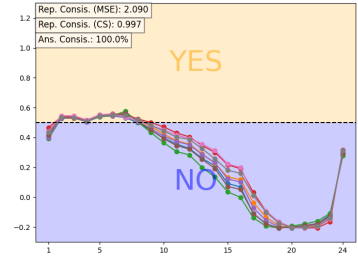
(c) TS2.1



(d) TS2.2



(e) TS2.3



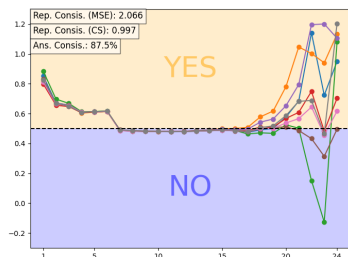
(f) TS2.4

Input: Although both operations were coordinated from Moscow, there was a shocking difference between their outcomes. Operation West was conducted in West Ukraine by the Soviet NKVD and targeted the families of suspected UPA members.

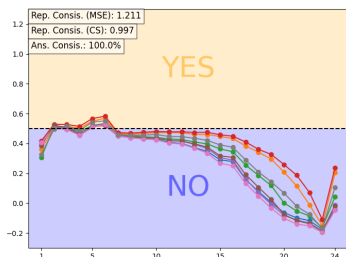
Cause: coordinated

Effect: targeted

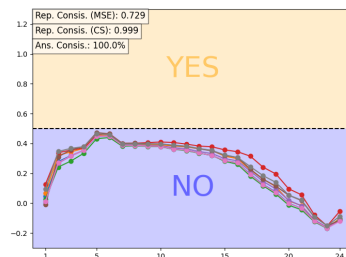
Label: no



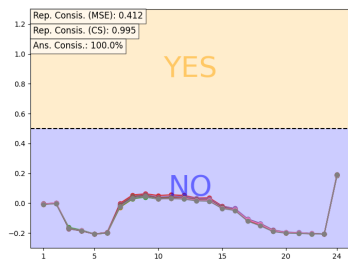
(a) TS0



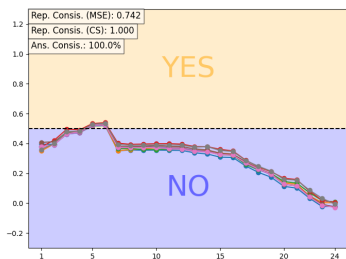
(b) TS1



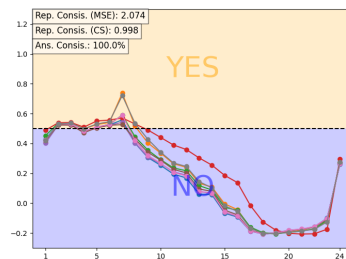
(c) TS2.1



(d) TS2.2



(e) TS2.3



(f) TS2.4